

Installation and data loading

Installation and Setup of Microsoft SQL Server and Azure Data Studio on Ubuntu (22.04)

1. Installation of Microsoft SQL Server Tools (CLI)

1.1 Adding the Microsoft SQL Tools Repository

The Microsoft package repository was added to the system in order to install SQL Server command-line tools.

```
curl https://packages.microsoft.com/keys/microsoft.asc | sudo tee /etc/apt/trusted.gpg.d/microsoft.asc
```

```
curl https://packages.microsoft.com/config/ubuntu/22.04/prod.list | sudo tee /etc/apt/sources.list.d/msprod.list
```

Update the package index:

```
sudo apt update
```

1.2 Installing mssql-tools and Dependencies

The SQL Server command-line client (sqlcmd) and required ODBC libraries were installed:

```
sudo ACCEPT_EULA=Y apt install -y mssql-tools unixodbc-dev
```

1.3 Adding sqlcmd to the System PATH

To make sqlcmd available globally:

```
echo 'export PATH="$PATH:/opt/mssql-tools/bin"' >> ~/.bashrc  
source ~/.bashrc
```

2. SQL Server Authentication and Configuration

2.1 Changing the SA Password

The system administrator (SA) password was set or changed using:

```
sudo /opt/mssql/bin/mssql-conf set-sa-password  
sudo systemctl restart mssql-server
```

2.2 Logging into SQL Server via CLI

To connect to SQL Server from the terminal:

```
sqlcmd -S localhost -U sa -P 'your-password'
```

Note: If -P is omitted, SQL Server prompts for the password interactively.

2.3 Verifying Login

Inside the sqlcmd prompt:

```
SELECT SYSTEM_USER;  
GO
```

Expected output:

```
sa
```

2.4 Checking SQL Server Version

```
SELECT @@VERSION;  
GO
```

3. **TPC-H Schema** and Data Overview

3.1 Schema Files Provided in the Repository

The repository contains the following schema-related files:

schema/tpch.sql

- Creates all TPC-H tables
- Loads data using BULK INSERT from .tbl files

schema/tpch_fk.sql

- Adds primary key and foreign key constraints
 - Optional but recommended for correctness and query optimization
-

3.2 Data Files Generated by dbgen

After running:

```
cd dbgen
make
./dbgen -s 1
```

The following flat files are generated:

- customer.tbl
- orders.tbl
- lineitem.tbl
- part.tbl
- partsupp.tbl
- supplier.tbl
- nation.tbl
- region.tbl

These files contain raw data and are not connected to SQL Server until explicitly loaded.

4. Loading TPC-H Data into SQL Server

4.1 Copying .tbl Files to SQL Server Data Directory

SQL Server can only read data from specific directories. The data files were copied as follows:

```
sudo mkdir -p /var/opt/mssql/data/tpch
sudo cp dbgen/*.tbl /var/opt/mssql/data/tpch/
sudo chown -R mssql:mssql /var/opt/mssql/data/tpch
```

4.2 Creating the TPC-H Database

Using either non-interactive or interactive mode:

```
CREATE DATABASE TPCH;
GO
```

Or from the shell:

```
sqlcmd -S localhost -U sa -P 'YourPassword' -Q "CREATE DATABASE TPCH"
```

4.3 Running the Schema and Data Load Script

```
sqlcmd -S localhost -U sa -P 'YourPassword' -d TPCH -i schema/tpch.sql
```

This script:

- Creates all TPC-H tables
- Loads data from the .tbl files using BULK INSERT

Important:

Before running the script, the file paths inside schema/tpch.sql were verified:

```
BULK INSERT customer
FROM '/var/opt/mssql/data/tpch/customer.tbl'
WITH (FIELDTERMINATOR='|', ROWTERMINATOR='|\\n');
```

Paths must match the actual data location.

4.4 Adding Primary and Foreign Keys (Optional)

```
sqlcmd -S localhost -U sa -P 'YourPassword' -d TPCH -i schema/tpch_fk.sql
```

This ensures schema integrity and enables better query planning.

5. Verifying Data Load

5.1 Connecting to TPCH Database

```
sqlcmd -S localhost -U sa -P 'YourPassword' -d TPCH
```

5.2 Checking Table Counts

```
SELECT COUNT(*) FROM lineitem;
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM customer;
```

GO

Expected row counts for Scale Factor 1:

- lineitem $\approx$ 6,000,000
- orders $\approx$ 1,500,000
- customer $\approx$ 150,000

5.3 Listing Tables

```
SELECT name FROM sys.tables;  
GO
```

5.4 Describing a Table

```
EXEC sp_help 'region';  
GO
```

6. Running SQL via Command Line

6.1 Non-Interactive Mode (Recommended for Scripts)

Run a SQL file:

```
sqlcmd -S localhost -U sa -P 'YourPassword' -d TPCH -i file.sql
```

Run a single query:

```
sqlcmd -S localhost -U sa -P 'YourPassword' -Q "SELECT name FROM sys.databases"
```

6.2 Interactive Mode

```
sqlcmd -S localhost -U sa -P 'YourPassword'
```

Inside the prompt:

```
USE TPCH;  
GO
```

Exit:

EXIT

7. Installing Azure Data Studio

7.1 Download

Azure Data Studio was downloaded from:

<https://learn.microsoft.com/sql/azure-data-studio/download-azure-data-studio>

For Ubuntu / Pop!_OS, the .deb package was used.

7.2 Installing the Package

```
cd ~/Downloads  
sudo dpkg -i azuredatstudio*.deb
```

7.3 Fixing Dependencies

```
sudo apt --fix-broken install
```

7.4 Launching Azure Data Studio

From terminal:

```
azuredatstudio
```

Or via application menu.

8. Connecting Azure Data Studio to SQL Server

8.1 Creating a New Connection

Connection settings:

Field	Value
Connection type	Microsoft SQL Server
Server	localhost
Authentication	SQL Login
Username	sa
Password	SA password
Database	TPCH (optional)

8.2 Enabling Execution Plans

Either:

- Click **Run with Actual Plan**, or
- Execute:

```
SET STATISTICS PROFILE ON;  
GO
```

8.3 Running a Test Query

```
SELECT c.c_name, l.l_shipdate  
FROM customer c  
JOIN orders o ON c.c_custkey = o.o_custkey  
JOIN lineitem l ON l.l_orderkey = o.o_orderkey;
```

This displays:

- Graphical execution plan
 - Join operators
 - Index scans and seeks
 - Estimated vs actual rows
-
-

Installation and Setup of Microsoft SQL Server Docker version:

```
# Pull SQL Server 2022 image  
docker pull mcr.microsoft.com/mssql/server:2022-latest  
  
# Run container  
docker run -e "ACCEPT_EULA=Y" \  
           -e "SA_PASSWORD=YourStrong@Pass123" \  
           -p 1433:1433 \  
           --name sqlserver2022 \  
           -d mcr.microsoft.com/mssql/server:2022-latest
```

Use the following commands to run the Docker container (you will be prompted to enter your password):

```
sudo systemctl start docker
docker start sqlserver2022
docker exec -it sqlserver2022 bash
sqlcmd -S localhost -U sa -C
```

Inside sqlcmd, create the **tpch** database and load the data (refer Section 4).

A Microsoft SQL Server instance running in Docker can be connected to Microsoft Azure using tools like Azure Data Studio via the exposed port (e.g., localhost:1433). Refer to Section 8.1 for the connection settings, where the exposed port number should be specified in the server field to connect to the Docker container.

User and Login creation

User and Login Creation for Row-Level Security in SQL Server

1. Introduction

Row-Level Security (RLS) in Microsoft SQL Server restricts access to rows in a table based on the identity of the user executing the query. To effectively implement and test RLS policies, it is necessary to create multiple users and simulate how different users interact with the database.

This section describes the process of creating server-level logins and database-level users, assigning permissions, and validating access. These steps form the foundation required to enforce and evaluate RLS policies.

2. Authentication vs Authorization

SQL Server distinguishes between authentication and authorization through two key entities:

- **Login (Authentication):**
A login is a server-level principal that allows a user to connect to the SQL Server instance.
- **User (Authorization):**
A user is a database-level principal associated with a login. It defines what actions the login can perform within a specific database.

Thus:

- A login enables access to the server.
- A user enables access within a database.

3. Creation of Server-Level Logins

To allow multiple identities to connect to SQL Server, three logins were created: user1, user2, and user3.

```
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'user1')
    CREATE LOGIN user1 WITH PASSWORD = 'User1@123', CHECK_POLICY = OFF;
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'user2')
```

Explanation:

- The CREATE LOGIN statement creates a server-level login.
- The IF NOT EXISTS condition ensures that duplicate logins are not created.
- CHECK_POLICY = OFF disables password policy enforcement, which simplifies setup for testing purposes.

Verification:

The system catalog view sys.server_principals was used to verify the existence of these logins:

```
SELECT name
FROM sys.server_principals
WHERE name IN ('user1','user2','user3');
GO
```

4. Creation of Database-Level Users

After creating logins, corresponding users were created within the database and mapped to these logins.

```
-- Database-level users (authorization)
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'user1')
    CREATE USER user1 FOR LOGIN user1;
```

Explanation:

- CREATE USER ... FOR LOGIN maps a login to a database user.
- This step is essential because a login alone cannot access database objects without a corresponding user.

Verification:

The system catalog view sys.database_principals was used to confirm successful creation:

```
SELECT name
FROM sys.database_principals
WHERE name IN ('user1','user2','user3');
GO
```

5. Granting Permissions

To enable users to query data, appropriate permissions were granted.

```
GRANT SELECT ON lineitem TO user1;
```

Explanation:

- **GRANT SELECT ON** lineitem allows users to read data from the lineitem table.
- GRANT SELECT ON SCHEMA::dbo provides broader access to objects within the schema.
- These permissions are necessary for users to execute queries that will be filtered by RLS policies.

6. Granting SHOWPLAN Permission

```
GRANT SHOWPLAN TO user1;
```

Explanation:

- The SHOWPLAN permission allows users to view execution plans.
- This is useful for analyzing query behavior and understanding how RLS impacts execution.

7. Testing Using User Context Switching

To simulate queries from different users without logging in separately, the EXECUTE AS statement was used.

```
EXECUTE AS USER = 'user1';  
SELECT COUNT(*) AS rows_seen_by_user1 FROM lineitem;  
REVERT;  
GO
```

Explanation:

- EXECUTE AS USER temporarily switches the execution context to the specified user.
- This allows testing of how RLS policies affect data visibility.
- REVERT restores the original execution context.

8. Role in Row-Level Security

The creation of logins and users is a prerequisite for implementing RLS. RLS policies rely on functions such as USER_NAME() to determine the identity of the current user.

Based on this identity, SQL Server applies filtering logic to restrict access to specific rows. Without properly configured users, it would not be possible to test or validate RLS behavior.

The code for the following can be found here:

- Creation of login: [create_login.sql](#)
- Creation of users: [create_users.sql](#)
- Grant permissions: [grant_permissions.sql](#)

RLS in MSSql

Row-Level Security (RLS) in Microsoft SQL Server

Row-Level Security (RLS) in Microsoft SQL Server is a feature that restricts data access at the row level based on the user executing the query. It ensures that users can only view or modify data they are authorized to access.

As part of this work, we converted 50 PostgreSQL queries to Microsoft SQL Server.

Links: [PostgreSQL](#) | [MSSQL](#)

1. How RLS is Enabled

RLS is implemented using **security policies** that enforce filtering logic on database tables.

1.1: Create a Predicate Function

A predicate function defines the access condition:

```
CREATE FUNCTION schema_name.function_name
(
    @column1 datatype,
    @column2 datatype
)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
(
    SELECT 1 AS result
    WHERE <filter_condition>
);
```

- This function returns rows only when the condition is satisfied
 - It acts as a filter applied automatically during query execution
-

1.2: Create a Security Policy

The predicate function is attached to a table using a security policy:

```
CREATE SECURITY POLICY policy_name
ADD FILTER PREDICATE schema_name.function_name(column1, column2)
ON table_name
WITH (STATE = ON);
```

- **FILTER PREDICATE** → restricts rows during SELECT, UPDATE, DELETE
 - **STATE = ON** → enables the policy
-

1.3: Testing

1.3.1 Clear Cache (Fair Measurement)

Before testing, the cache is cleared to ensure accurate and unbiased performance results:

```
DBCC FREEPROCCACHE;
DBCC DROPCLANBUFFERS;
GO
```

- Removes execution plans and buffer cache
- Ensures queries are executed from scratch

1.3.2 Enable Performance Metrics

```
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO
```

- Measures query execution time
- Tracks logical and physical I/O operations

1.3.3 Use Query Recompilation

```
OPTION (RECOMPILE);
```

- Forces SQL Server to generate a fresh execution plan
- Prevents reuse of cached plans

1.3.4 Execute Queries

```
-- Enable
ALTER SECURITY POLICY policy_name WITH (STATE = ON);

-- Disable
ALTER SECURITY POLICY policy_name WITH (STATE = OFF);
```

- Run queries (e.g., `SELECT COUNT(*)`)
- Test in two scenarios:
 - With RLS disabled
 - With RLS enabled

This allows performance comparison and controlled testing.

Link - [test script](#)

2. How Policies Work

- When a query is executed, SQL Server automatically invokes the predicate function
- The function evaluates conditions for each row
- Only rows satisfying the condition are returned

The filtering is **transparent**, meaning:

- No change is required in application queries
 - Security is enforced at the database level
-

3. Role of User in RLS

In Row-Level Security (RLS) in Microsoft SQL Server, access control can be defined based on the current executing user using the `USER_NAME()` function, which returns the active database user. This allows certain users, such as user3, to be treated as privileged and granted full access to all rows, while other users are restricted according to specific conditions

The policy includes a check where if the current user matches the privileged user (user3), all data is accessible; otherwise, only rows satisfying the defined filtering condition are returned.

4. Difference between PostgreSQL and MSSql

Row-Level Security (RLS) differs significantly between PostgreSQL and Microsoft SQL Server in terms of implementation and flexibility. In PostgreSQL, RLS is enabled per table, policies are directly bound to roles, multiple policies can be defined on a single table, and features such as **ILIKE**, **INTERVAL**, and **FORCE ROW LEVEL SECURITY** are supported, while table owners bypass RLS by default. In contrast, SQL Server implements RLS using security policies that are not directly bound to users or roles; instead, user-specific logic must be handled within predicate functions. SQL Server typically uses a single security policy per table, does not support **ILIKE** (and case sensitivity in **LIKE** depends on collation), replaces **INTERVAL** with **DATEADD**, and does not provide a **FORCE RLS** feature, while highly privileged roles such as sysadmin can bypass RLS.

Certain PostgreSQL capabilities, such as direct policy-to-role binding, multiple independent policies per table, and forced RLS enforcement, are not available in SQL Server; however, similar behavior can be achieved through workarounds like using **USER_NAME()** within predicate functions and combining multiple conditions within a single policy.

5. Scalar Subqueries and Their Limitations in RLS

In the context of Row-Level Security (RLS) in Microsoft SQL Server, subqueries are often used to define access conditions. A scalar subquery is a subquery that returns exactly one value (one row and one column) and is commonly used in comparisons. For example:

```
SELECT *
FROM customer c
WHERE c.c_acctbal >
(
    SELECT AVG(c2.c_acctbal)
    FROM customer c2
);
```

In this case, the subquery returns a single value (the average account balance), and each row is compared against it, making it a scalar subquery. However, scalar subqueries can be problematic in RLS because they must always return exactly one row; if multiple rows are returned, SQL Server raises an error, and if no rows are returned, the result becomes **NULL**, potentially leading to incorrect filtering. Therefore, in RLS predicate functions, scalar subqueries are generally avoided when only a boolean check is required. Instead, constructs like **EXISTS** are preferred, as they safely return a true/false result without the risk of errors. For example:

```
WHERE EXISTS
(
    SELECT 1
    FROM some_table
    WHERE condition
);
```

This makes **EXISTS** more suitable and reliable for implementing row-level access control.

Automation scripts

Experimental Framework for Evaluating Row-Level Security (RLS)

1. Introduction

To evaluate the performance and behavior of Row-Level Security (RLS) in Microsoft SQL Server, an experimental framework was developed to automate the execution of TPC-H benchmark queries under different RLS configurations while systematically collecting performance metrics.

The objective of these experiments is to:

- Execute benchmark queries under controlled RLS configurations
- Dynamically apply different security policies
- Compare performance with and without RLS
- Analyze how multiple predicates interact across tables
- Measure execution overhead using detailed statistics

The framework supports multiple execution strategies, ranging from exhaustive evaluation of predicate combinations to controlled and randomized experiments. It is configuration-driven, reproducible, and scalable, enabling both broad and targeted analysis of RLS behavior.

2. System Organization

The experimental setup is organized into multiple directories and configuration files to ensure modularity and scalability.

2.1 Query Set

- The [queries/](#) directory contains SQL files (1.sql to 22.sql) corresponding to the TPC-H benchmark queries.
- Each query represents a different workload scenario involving multiple tables.

2.2 Predicate Definitions

- The [predicates/](#) directory contains subfolders for each table.

Each JSON file represents a single RLS predicate definition. It contains:

- `policy_id`: Unique identifier for the policy
- `table`: Table on which the policy is applied
- `predicate_name`: Name of the predicate function

- columns: Columns passed as parameters to the predicate

```
1  {
2    "policy_id": "P14",
3    "table": "customer",
4    "predicate_name": "dbo.p14_customer_asia_balance",
5    "columns": ["c_acctbal", "c_nationkey"]
6  }
```

Purpose:

- Acts as a bridge between SQL predicate functions and Python scripts
- Allows dynamic loading and application of predicates
- Avoids hardcoding predicate logic inside scripts

These predicates define the logical conditions enforced by RLS.

2.3 Predicate Functions

- The [predicate_functions/](#) directory contains SQL scripts that define inline table-valued functions.
- These functions implement the filtering logic used in RLS policies.
- All predicate functions are *pre-created* before running experiments.

2.4 Policy Template

- The file [policy_template.sql](#) provides a reusable template for dynamically creating security policies.
- Policies are created and dropped during runtime using this template.

2.5 Configuration Files

Database Configuration ([db_config.json](#))

Defines:

- Server connection details
- Authentication credentials
- Test users
- Machine metadata for experiment tracking

Purpose:

- Centralizes database connection settings
- Enables switching environments without modifying code

- Stores experiment context (e.g., machine details for reproducibility)

Experiment Configuration ([experiment_config.json](#))

Controls:

- Number of runs per query
- Execution modes (WITH_RLS, WITHOUT_RLS)
- Predicate selection strategy
- Execution settings (statistics collection, delays)
- Output file configuration

Purpose:

- Controls experiment behavior without modifying scripts
- Ensures consistency across runs
- Enables easy tuning of experimental parameters

3. Experimental Design

The experiments are designed to evaluate RLS under different conditions by varying:

- Predicate selection strategy
- Number of policies applied
- Query workload
- Execution mode (RLS enabled vs disabled)

Each experiment:

1. Selects predicates
2. Dynamically creates security policies
3. Executes queries under a specific user context
4. Collects performance statistics
5. Stores results in a structured CSV file

4. Execution Modes and Strategies

4.1 Mode 1: Query with All Predicate Combinations (Cartesian per Query)

Script: [mode1_query_only.py](#)

Purpose

To evaluate the impact of all possible predicate combinations for a given query.

Methodology

- For a selected query:
 - Extract all tables used
 - Load all predicates for each table
 - Generate the Cartesian product of predicates across tables
- For each combination:
 - Apply one predicate per table
 - Execute query under both:
 - WITH_RLS (policies enabled)
 - WITHOUT_RLS (policies disabled)
- Repeat execution multiple times

Example

Query uses tables: customer, orders

Predicates:

- customer $\rightarrow$ P1, P2
- orders $\rightarrow$ P3, P4

Combinations:

- (P1, P3)
- (P1, P4)
- (P2, P3)
- (P2, P4)

Each combination applies **both predicates together**.

How to Run

```
python3 -m experiments.mode1_query_only 3.sql
```

4.2 Mode 2: Query with Explicitly Selected Predicates

Script: [mode2_query_with_predicates.py](#)

Purpose

To allow controlled experiments using manually selected predicates.

Methodology

- User specifies:
 - Query name
 - Predicate for each table (via command-line arguments)
- For the given selection:
 - Create corresponding policies
 - Execute query under both modes
 - Record results

Example

```
python3 -m experiments.mode2_query_with_predicates 3.sql  
customer:p14_customer_asia_balance.json orders:p12_orders_discount.json
```

This applies:

- One predicate on customer
- One predicate on orders

4.3 Cartesian Strategy Across All Queries

Script: [cartesian.py](#)

Purpose

To evaluate the impact of multiple predicate combinations across all queries.

Methodology

- Iterate over all queries
- For each query:
 - Extract tables used
 - Load predicates per table
 - Generate the Cartesian product of predicates across tables
- Each combination consists of:
 - Exactly one predicate per table
- For each combination:
 - All selected predicates are applied simultaneously
 - The query is executed under both RLS-enabled and RLS-disabled modes

Example

If Query 5 uses:

- `lineitem` → P1, P2
- `orders` → P3

Combinations:

- (P1, P3)
- (P2, P3)

How to Run

```
python3 -m experiments.cartesian.py
```

4.4 No-Cartesian Strategy (Single Predicate per Table)

Script: [no_cartesian.py](#)

Purpose

To evaluate RLS performance without combinatorial explosion.

Methodology

- For each query:
 - Randomly select one predicate per table
- Apply selected predicates
- Execute query under both modes
- Repeat for multiple runs

Example

Query uses:

- `customer` → randomly picks P2
- `orders` → randomly picks P3

Applied combination:

- (P2, P3)

Only this combination is tested.

How to Run

```
python3 -m experiments.no_cartesian.py
```

5. Policy Creation Mechanism

Policies are dynamically created using a template:

```
1 DROP SECURITY POLICY IF EXISTS dbo.{policy_name};
2 GO
3
4 CREATE SECURITY POLICY dbo.{policy_name}
5 ADD FILTER PREDICATE {predicate_name}({columns})
6 ON dbo.{table}
7 WITH (STATE = {state});
8 GO
```

Process

- For each predicate:
 - Substitute values into the template
 - Create or replace the security policy
- STATE = ON enables RLS
- STATE = OFF disables RLS without dropping the policy

This allows direct comparison between RLS-enabled and RLS-disabled executions.

6. Query Execution Framework

All queries are executed using a standardized wrapper:

```
def execute_query_as_user(query_sql, user):
    wrapped = f"""
    CHECKPOINT;
    DBCC DROPCLEANBUFFERS;
    EXECUTE AS USER = '{user}';
    SET STATISTICS IO ON;
    SET STATISTICS TIME ON;
    {query_sql}
    REVERT;
    """
    return run_sql(wrapped, DB_CONF)
```

Purpose of Each Step

- CHECKPOINT and DBCC DROPCLEANBUFFERS: reduce caching effects
- EXECUTE AS USER: simulate user-specific RLS behavior
- SET STATISTICS IO/TIME: collect performance metrics
- REVERT: restore original execution context

7. Performance Metrics Collected

For each execution, the following metrics are recorded:

- CPU time (cpu_ms)
- Elapsed time (elapsed_ms)
- Logical reads
- Number of rows returned
- Errors (if any)

Additional metadata:

- Query name
- Tables involved
- Applied policies
- Execution mode
- User
- Machine details

8. Output Format

Results are stored in CSV format with all the metrics collected:

```
mode, strategy, run_number, user, policy_ids, predicate_functions, tables_affected, cpu_ms, elapsed_ms, logical_reads, rows_returned, errors, machine, query_name, tables_used
WITH RLS, MODEL CARTESIAN, 1, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 644, 66, 298871, 0, Mr, 3, sql, "lineitem, orders, customer"
WITH RLS, MODEL CARTESIAN, 2, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 750, 80, 298871, 0, Mr, 3, sql, "lineitem, orders, customer"
WITH RLS, MODEL CARTESIAN, 3, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 688, 73, 298871, 0, Mr, 3, sql, "lineitem, orders, customer"
WITH RLS, MODEL CARTESIAN, 4, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 822, 85, 291680, 0, Mr, 3, sql, "lineitem, orders, customer"
WITH RLS, MODEL CARTESIAN, 5, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 582, 61, 291680, 0, Mr, 3, sql, "lineitem, orders, customer"
NO RLS, MODEL CARTESIAN, 1, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 737, 75, 131412, 11620, Mr, 3, sql, "lineitem, orders, customer"
NO RLS, MODEL CARTESIAN, 2, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 978, 100, 131412, 11620, Mr, 3, sql, "lineitem, orders, customer"
NO RLS, MODEL CARTESIAN, 3, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 1050, 104, 131412, 11620, Mr, 3, sql, "lineitem, orders, customer"
NO RLS, MODEL CARTESIAN, 4, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 998, 99, 131412, 11620, Mr, 3, sql, "lineitem, orders, customer"
NO RLS, MODEL CARTESIAN, 5, user1, P6;P13;P14, dbo.p6_ship_commit;dbo.p13_orders_customer_europe;dbo.p14_customer_asia_balance, lineitem;orders;customer, 998, 104, 131412, 11620, Mr, 3, sql, "lineitem, orders, customer"
```

This structured format enables further analysis and visualization.

Experiment results

Visualization and Analysis of Experimental Results

The results obtained from the experimental framework were analyzed using Python-based visualization scripts. These scripts process the generated CSV result files and produce plots that help interpret the impact of Row-Level Security (RLS) under different configurations.

All plots are generated from the experiment output files (CSV format) produced by the execution framework.

1. Performance Metrics and Cold-Cache Execution Model

1.1 Overview

To evaluate the impact of Row-Level Security (RLS) on query execution, performance metrics were collected using SQL Server's `SET STATISTICS IO` and `SET STATISTICS TIME`. These metrics capture different aspects of query execution, including CPU usage, memory access, and disk I/O.

To ensure that these measurements accurately reflect query behavior (especially disk I/O) each query was executed under a **cold-cache execution model**, where no previously cached data pages are reused. This allows consistent and fair comparison across runs and ensures that I/O metrics represent actual data access rather than cached results.

1.2 Cold Cache Execution Setup

Before each query execution, the buffer pool was cleared using:

```
CHECKPOINT;
```

```
DBCC DROPCLEANBUFFERS;
```

This setup ensures that:

- No data pages are reused from memory
- Each query behaves like a first-time execution
- Disk I/O metrics (especially read-ahead reads) are accurately captured

This approach is analogous to measuring **physical reads in Oracle**, where cache reuse is minimized.

Role of CHECKPOINT

- Flushes all **dirty pages** (modified pages in memory) to disk

- Ensures memory and disk are consistent
- Does not remove pages from memory

This step is required because only clean pages can be removed from the buffer pool.

Role of DBCC DROPCLEANBUFFERS

- Removes all **clean pages** from the buffer pool
- Forces subsequent queries to fetch data from disk

It does not:

- Delete data
- Affect database objects
- Clear execution plans

Why Both Are Required

- CHECKPOINT converts dirty pages into clean pages
- DBCC DROPCLEANBUFFERS removes those clean pages

Together, they ensure that the buffer pool is effectively cleared.

Effect on Query Execution

After applying both commands:

- All required data must be fetched from disk
- Query execution is independent of previous runs
- Disk I/O measurements reflect actual data access patterns

This is particularly important for correctly interpreting I/O-related metrics.

1.3 Metrics Collected

The following metrics were used in the plotting of the scripts:

- Execution time
- Logical reads
- Read-ahead reads
- Physical reads

Each metric captures a different aspect of query cost.

1.4 Execution Time

Total time taken to execute a query (in milliseconds).

- Represents end-to-end latency
- Includes CPU, memory, and I/O delays
- Reflects **user-visible performance**

1.5 Logical Reads

Number of data pages accessed from the buffer pool (memory).

- Indicates amount of data processed
- Reflects CPU and memory usage
- Higher values imply more extensive scanning or processing

1.6 Disk I/O: Physical Reads vs Read-Ahead Reads

SQL Server distinguishes between two types of disk access:

Physical Reads

- Synchronous disk I/O
- Occur when a required page is not in memory
- Query execution is blocked until the page is fetched
- Typically associated with random access patterns (e.g., index seeks)

Read-Ahead Reads

- Asynchronous, prefetch-based disk I/O
- SQL Server predicts future page accesses and loads them in advance
- Fetches multiple contiguous pages efficiently
- Common in scan-heavy queries

1.7 Key Relationship

Both metrics represent disk I/O:

$$\text{Total Disk I/O} = \text{physical_reads} + \text{read_ahead_reads}$$

They are reported separately because they reflect different access patterns:

- Physical reads → random, blocking I/O
- Read-ahead reads → sequential, optimized I/O

1.8 Observations in This Study

Under the enforced cold-cache model and TPC-H workload:

- Queries are predominantly scan-heavy
- Read-ahead reads dominate disk I/O
- Physical reads are often minimal

This indicates that SQL Server efficiently uses sequential prefetching for analytical queries.

1.9 Relation to Other Systems (Oracle Mapping)

For comparison with Oracle:

Oracle Metric	SQL Server Equivalent
Consistent gets	Logical reads
Physical reads	Read-ahead reads + physical reads

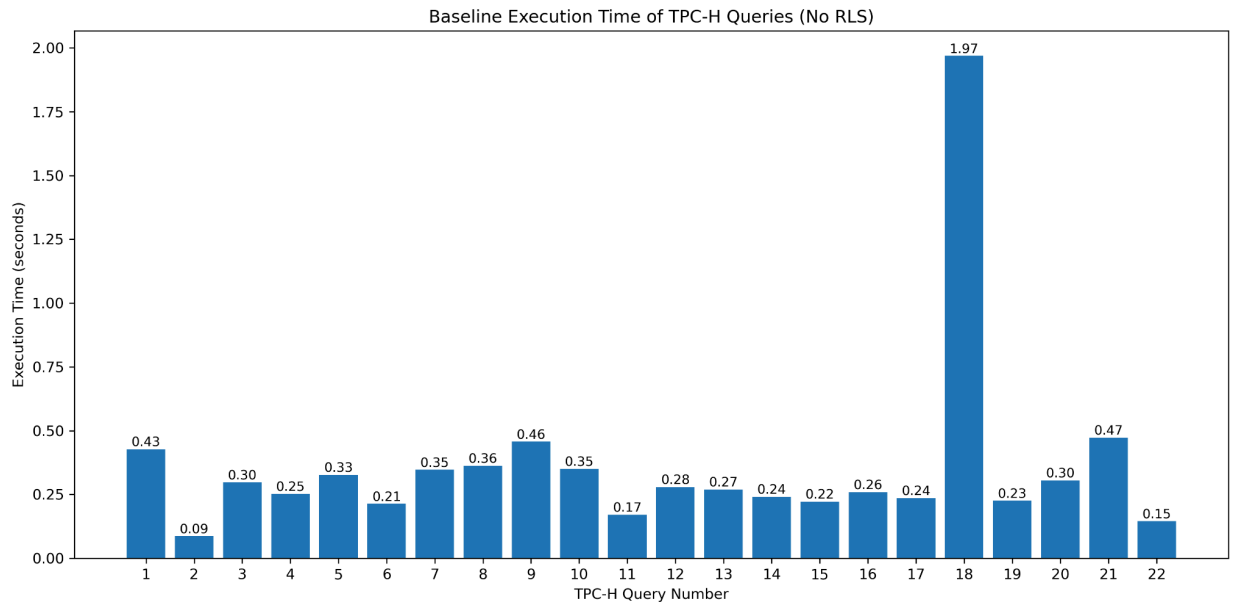
In analytical workloads, SQL Server read-ahead reads closely align with Oracle's physical read metric.

Using the above metrics (execution time, logical reads, read-ahead reads, and physical reads), plots were generated for each experimental category (baseline, single-policy, multi-policy, and query-specific analysis) to visually compare performance and understand the impact of RLS under different configurations.

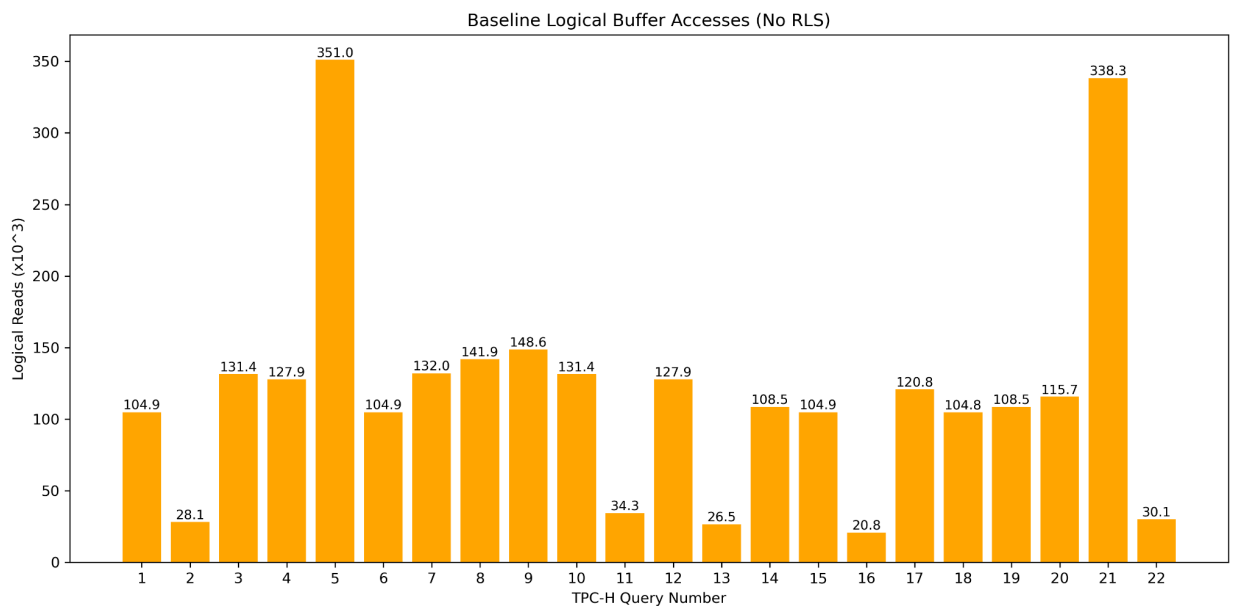
2. Baseline Performance (No RLS)

This set of plots represents the execution of all TPC-H queries **without any RLS policies applied**.

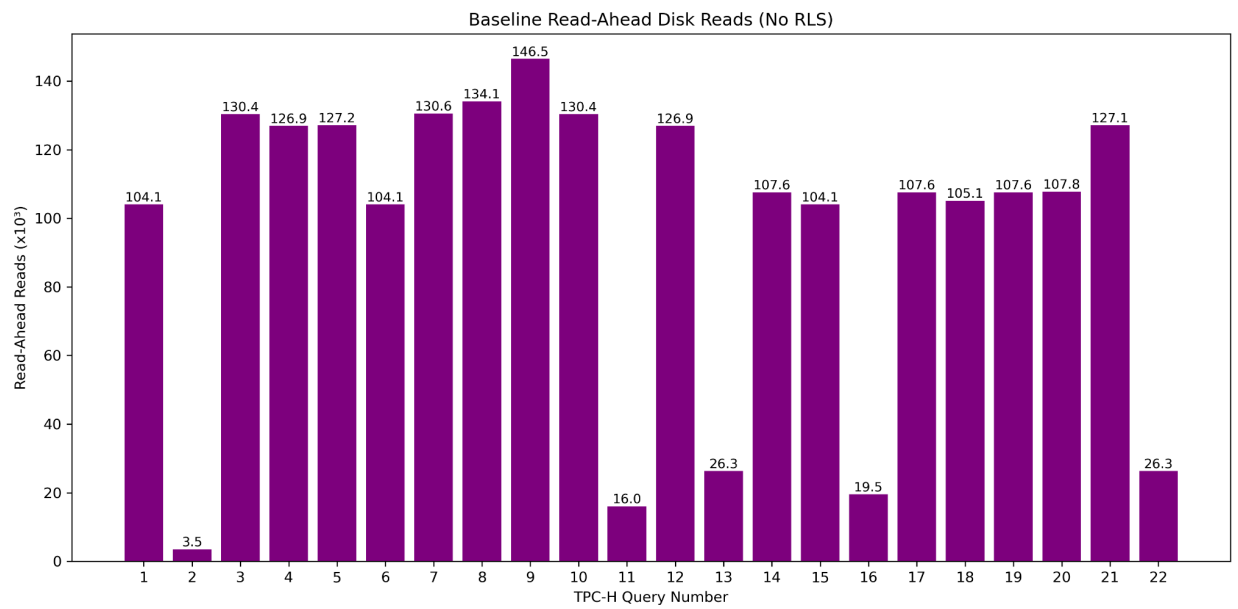
- Execution time for each query



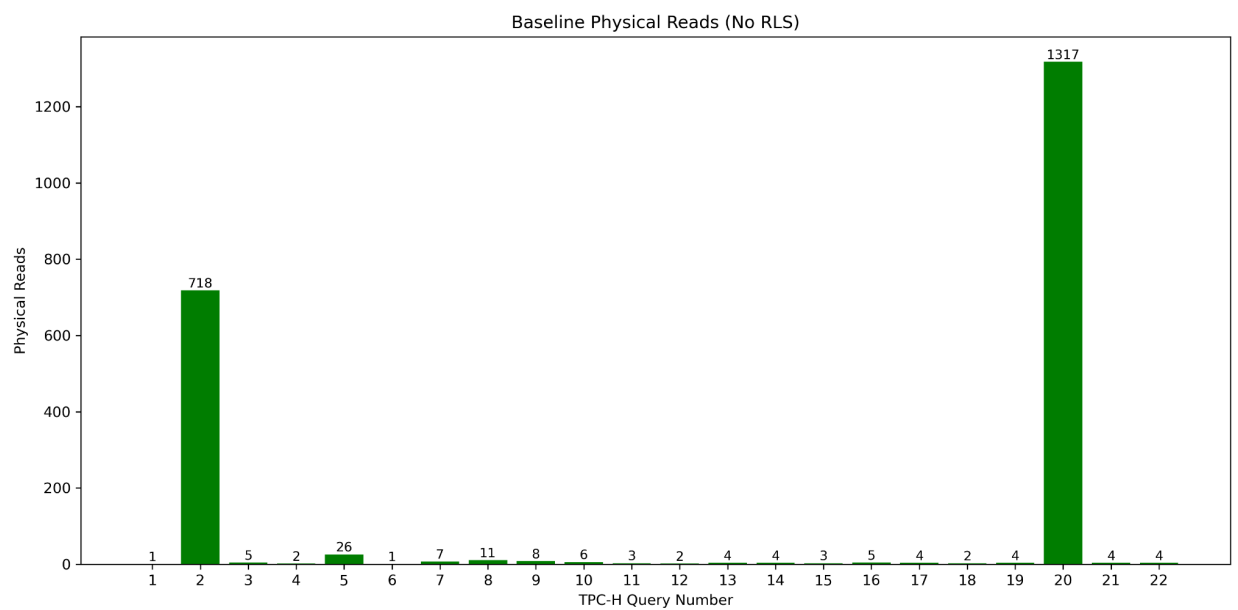
- Logical reads (buffer accesses)



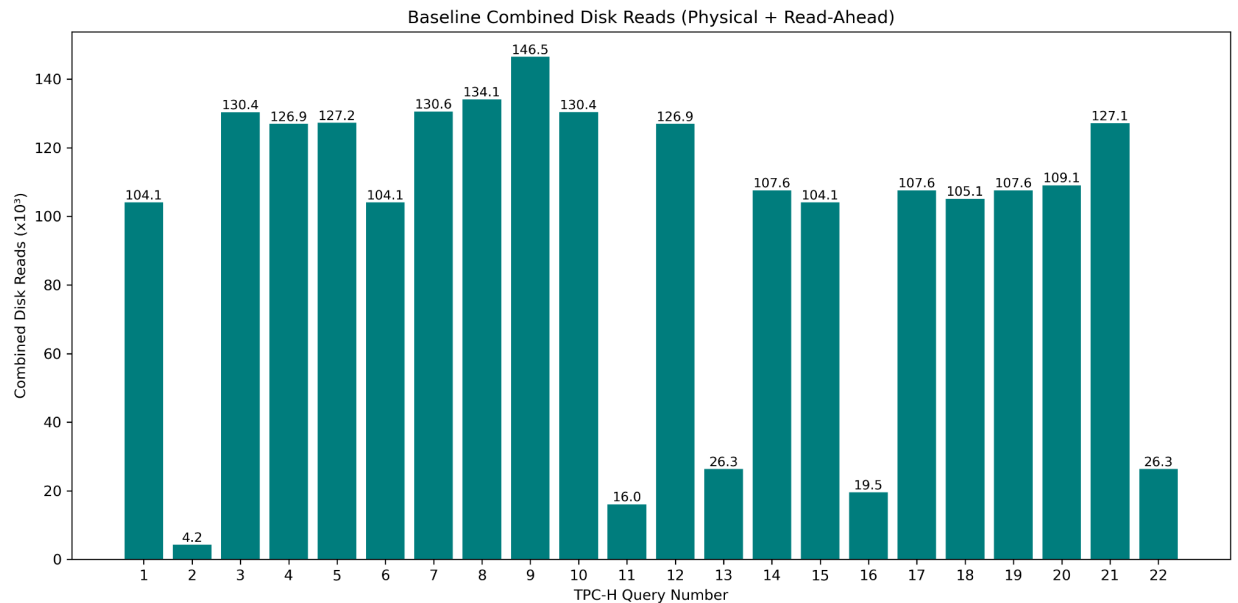
- Read-ahead reads



- Physical disk reads



- Combined disk reads



These plots:

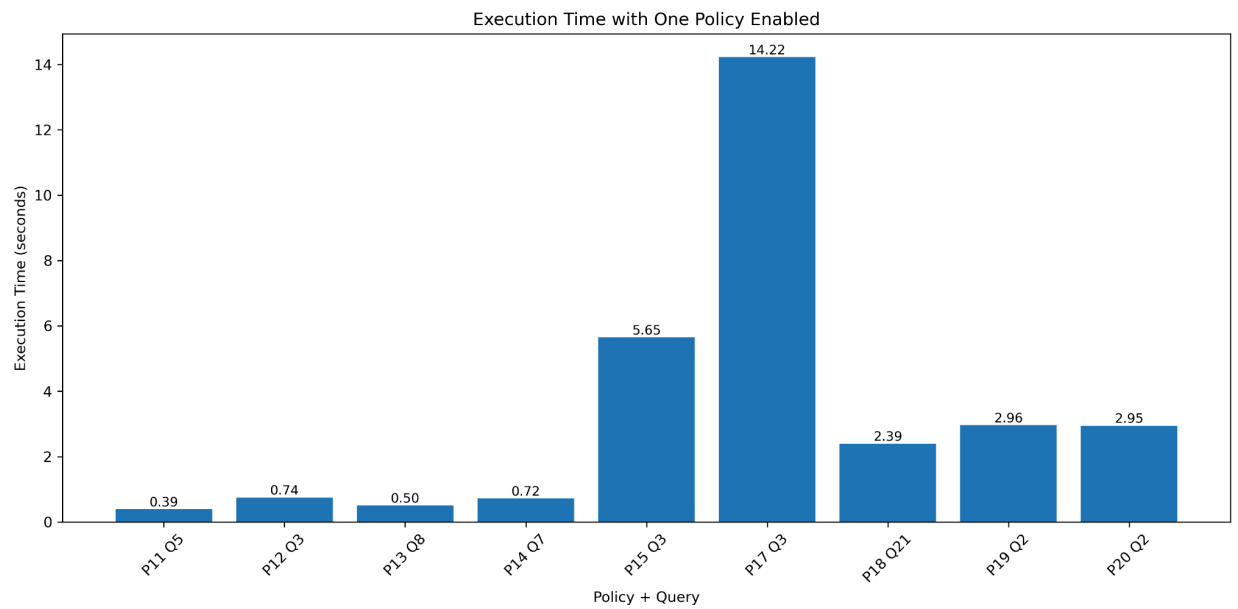
- Provide a **reference point** for comparison
- Help identify inherently expensive queries
- Serve as a baseline to measure RLS overhead

3. Single Policy per Query

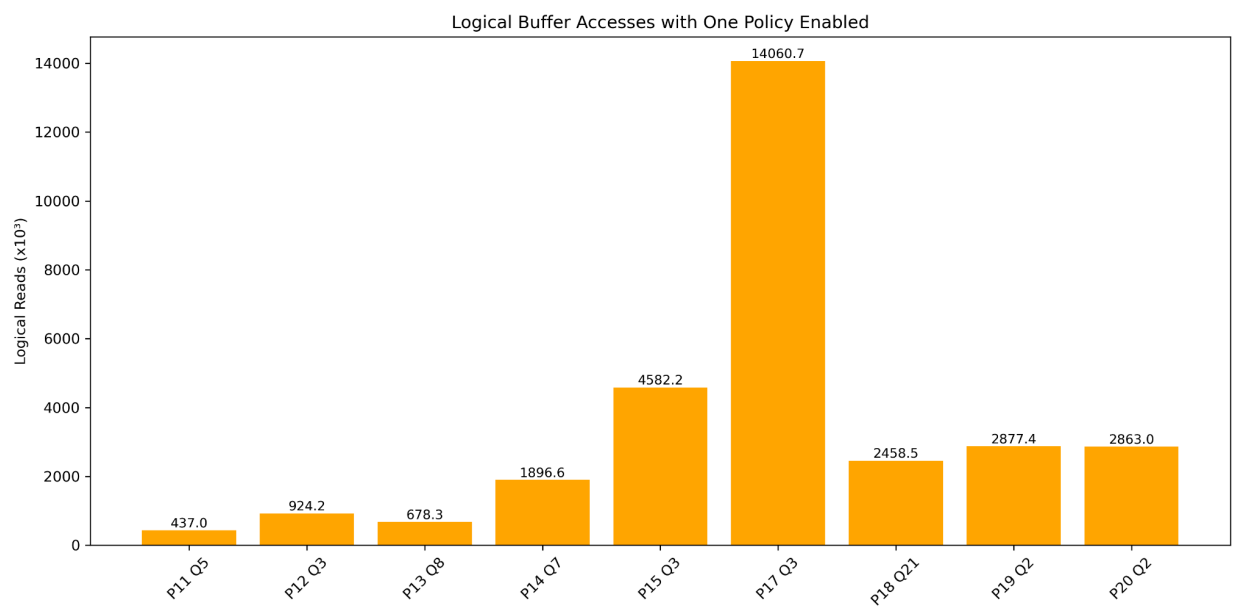
These plots represent experiments where **only one policy is applied to one query at a time**.

Each bar corresponds to a *(policy, query)* pair. Example label: P11 Q5 → Policy P11 applied on Query 5

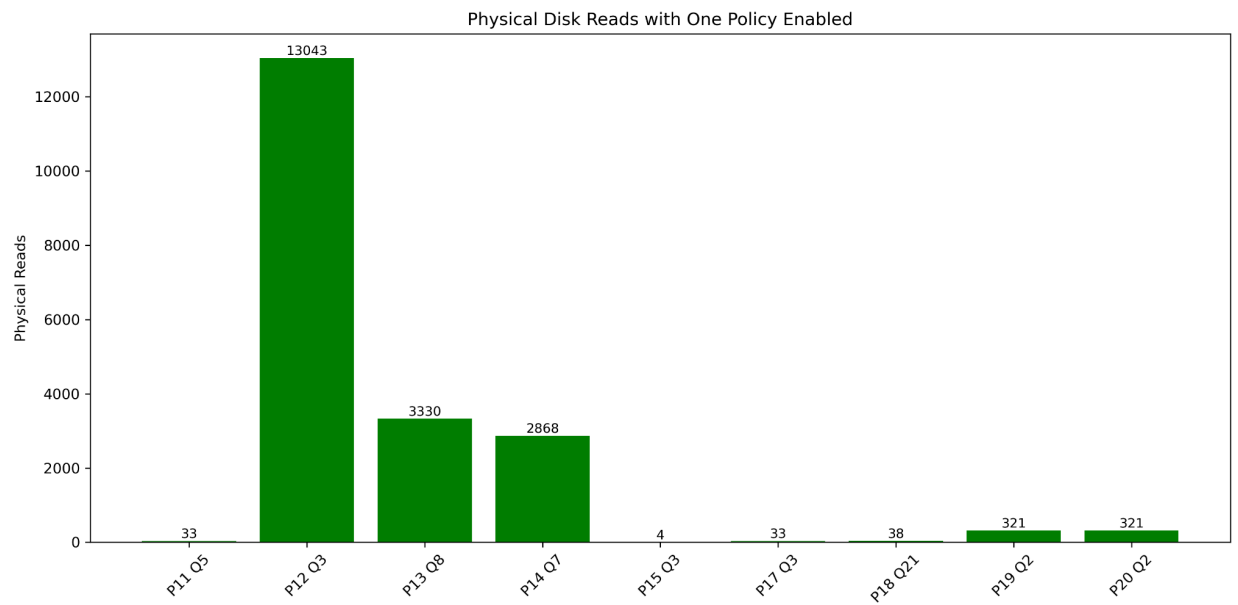
- Execution time



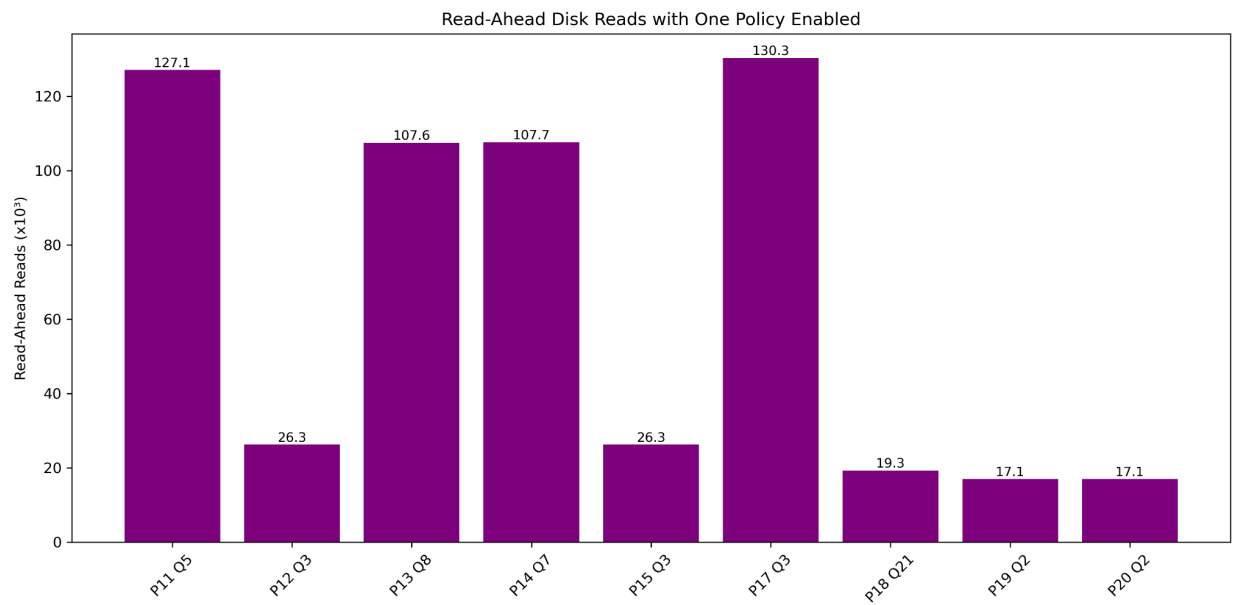
- Logical reads



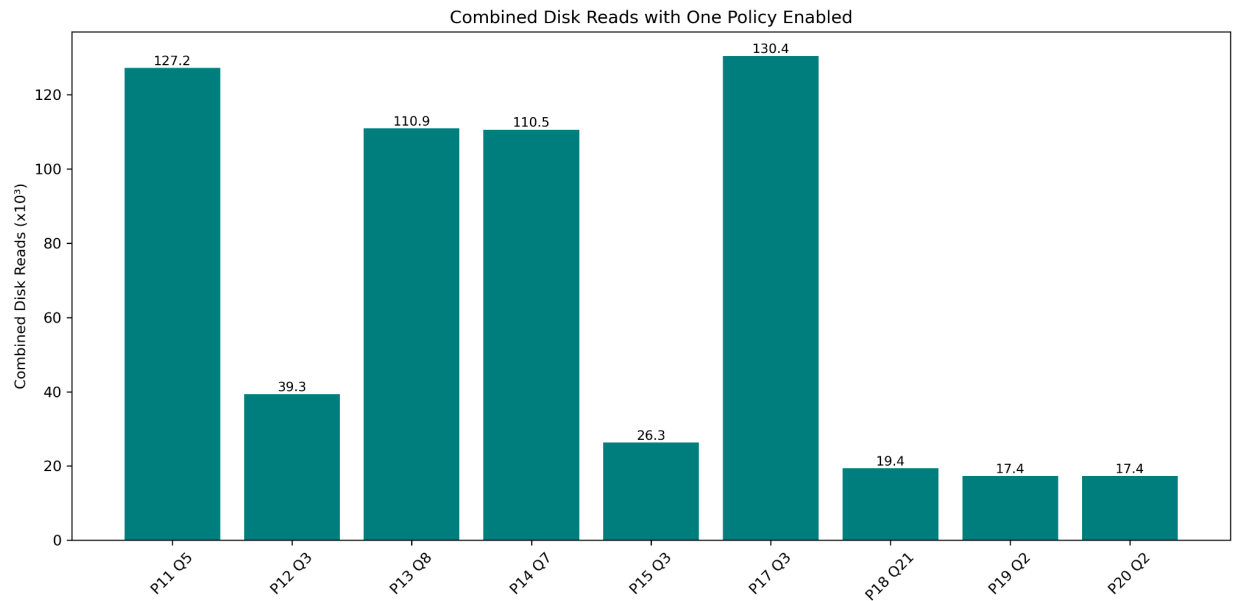
- Physical reads



- Read-ahead reads



- Combined reads

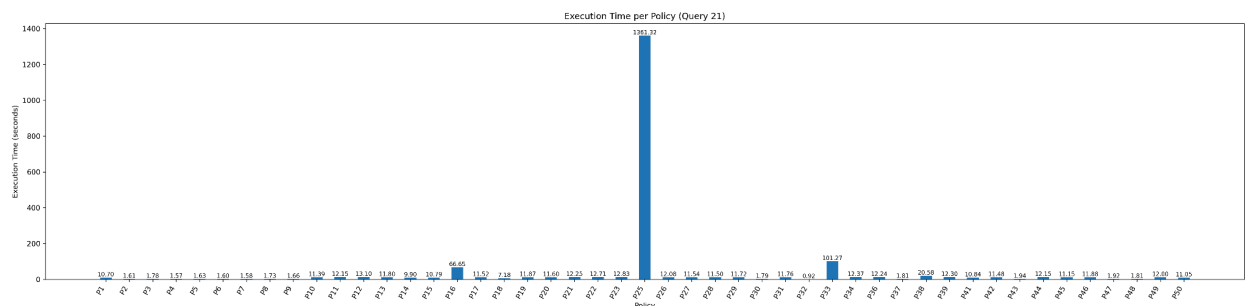


4. Per-Policy Analysis for a Single Query (Query 21)

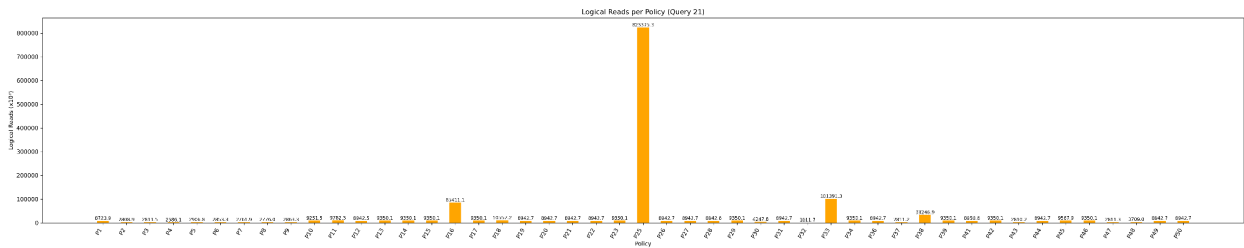
This set focuses on **Query 21**, where different policies are applied one at a time.

In the plots: X-axis: Policies (P1, P2, ...), Y-axis: Performance metric

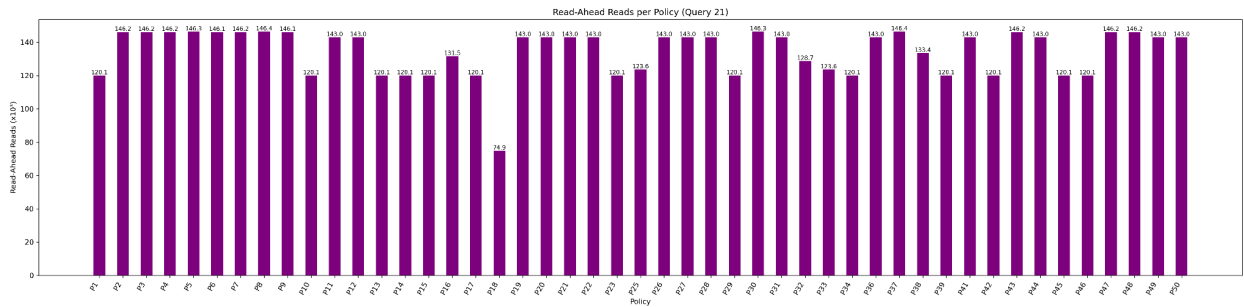
- Execution time



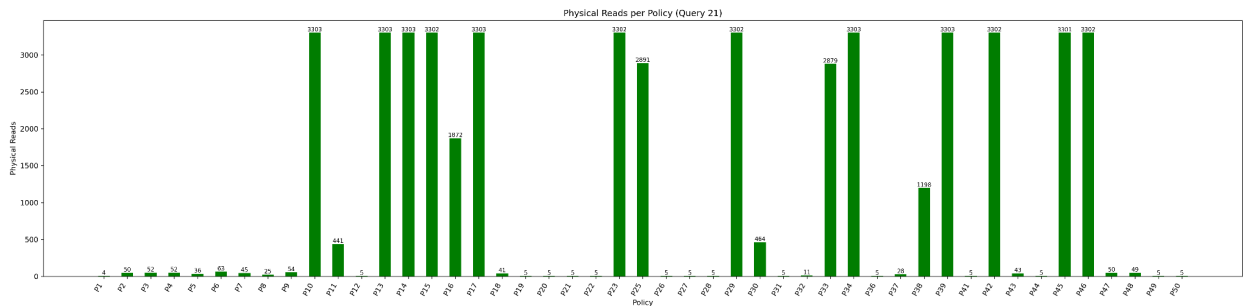
- Logical reads



- Read-ahead reads



- Physical reads



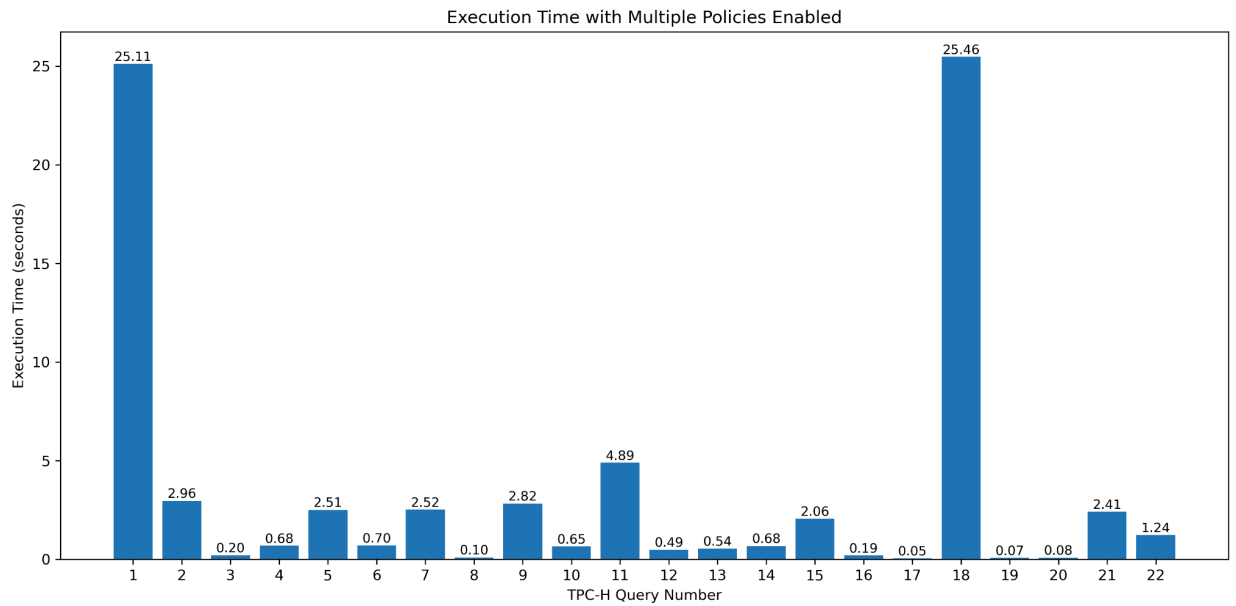
These plots enable direct comparison of policies for a **single query**

5. Selected Multiple Policies Enabled

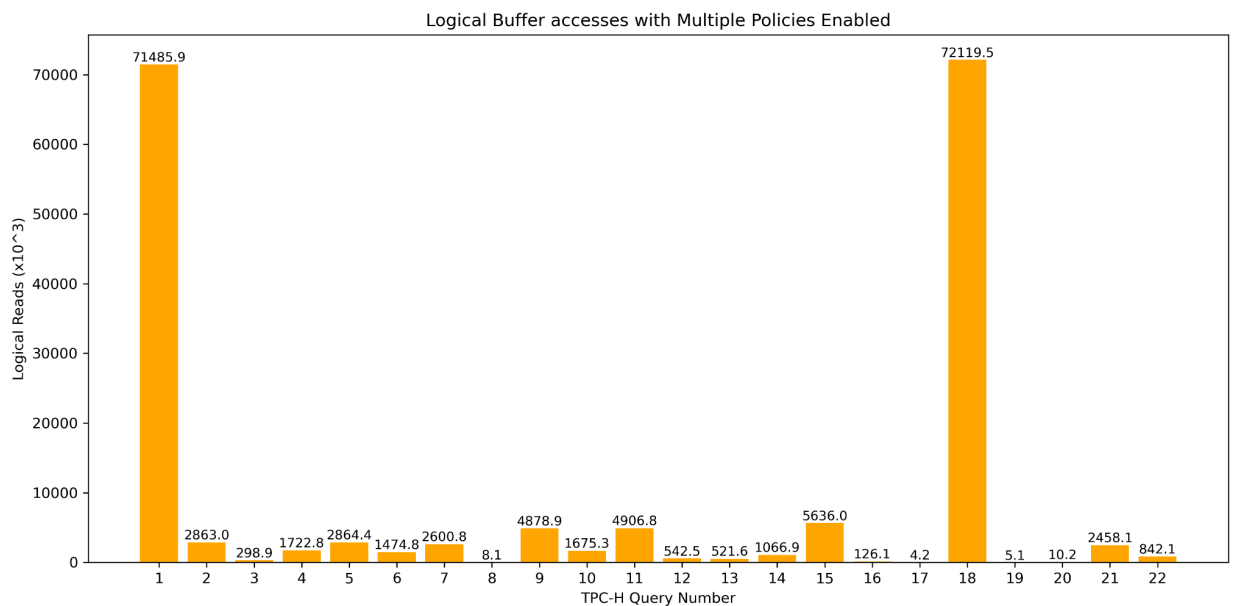
Description

These plots correspond to experiments where a **fixed set of policies (P11, P13, P14, P18, P20)** were applied simultaneously across all queries.

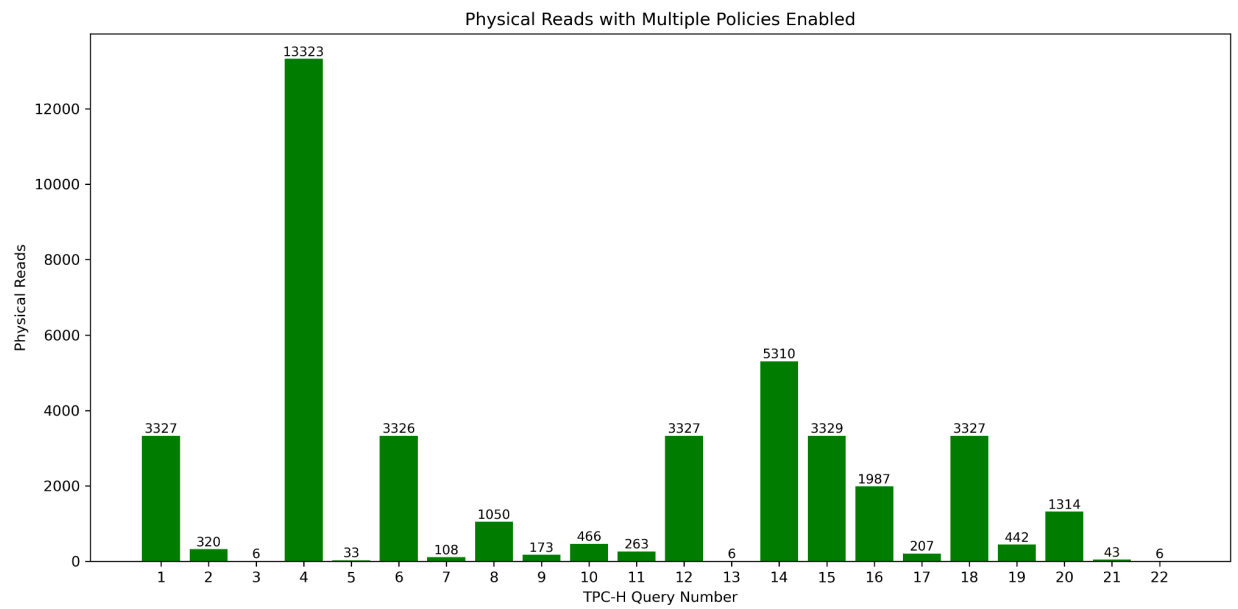
- Execution time across queries with multiple policies active



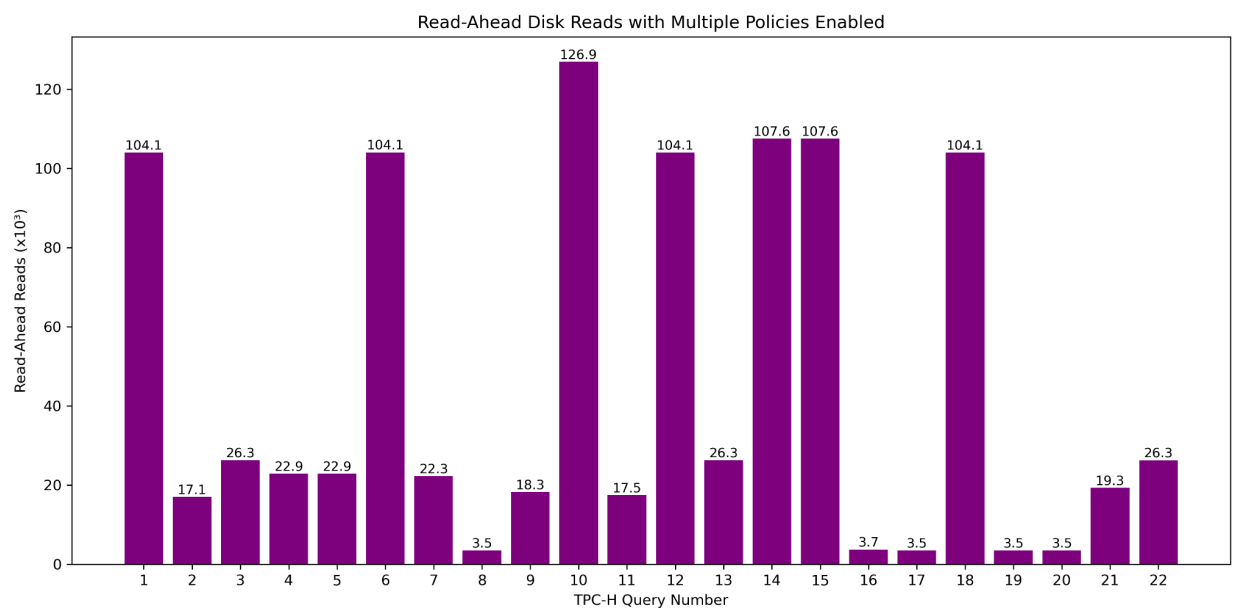
- Logical reads



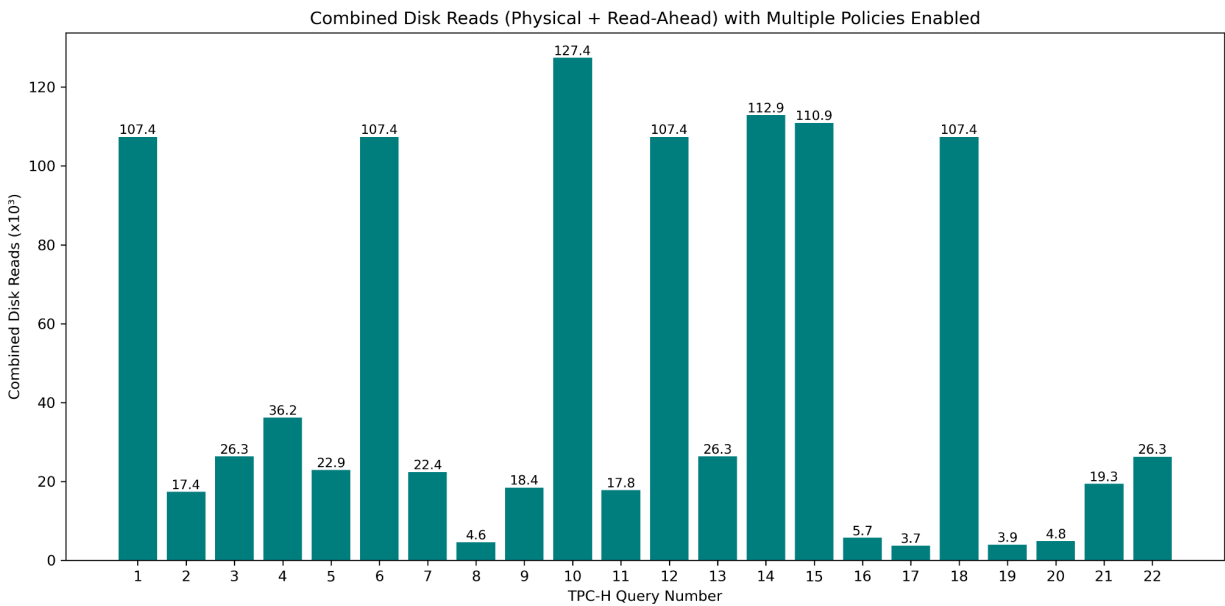
- Physical reads



- Read ahead reads



- Combined disk access behavior



6. Visualization Scripts

The following scripts were used to generate the plots:

- [plot_without_policy.py](#)
- [plot_policy_impact.py](#)
- [plot_one_policy_at_a_time.py](#)

Input to Scripts

- CSV files generated from the experiment runs ([results_all_queries.csv](#) , [results_One-Policy-at-a-Time_Evaluation.csv](#), [merged.csv](#) , [results/results_multiple_policies_simultaneously_enabled.csv](#))

These CSV files contain:

- Query execution metrics
- Applied policies
- Execution mode (WITH_RLS / WITHOUT_RLS)
- Performance statistics (time, I/O, etc.)

Execution Plan Operators

- Stream Aggregate
- Hash Aggregate
- Index Spool
- Assert

1. Index Spool

An Index Spool is a temporary indexed structure created during query execution to store intermediate results and speed up repeated data access.

- Not a permanent index
- Created dynamically during execution
- Stored in memory or tempdb

Example

Consider a query where a subquery is evaluated repeatedly for each row:

```
SELECT *
FROM orders o
WHERE EXISTS (
    SELECT 1
    FROM lineitem l
    WHERE l.l_orderkey = o.o_orderkey
);
```

- SQL Server scans the lineitem table once
- Builds a temporary index (spool)
- Reuses it for each orders row

Analysis:

- In this query Index Spool is created because the same table (lineitem) is accessed repeatedly
 - SQL Server builds the spool once and reuses it multiple times
 - Index Spool is generally created for non-SARGable queries (server does not use index seek to evaluate)
 - It happens in case of SARGable queries which are called multiple times like in the lineitem query.
-

2. Stream Aggregate

A Stream Aggregate computes aggregate results (e.g., COUNT, SUM, AVG) by processing sorted input data sequentially.

Example

```
SELECT l_suppkey, COUNT(*)
FROM lineitem
GROUP BY l_suppkey;
```

If data is already sorted on l_suppkey, SQL Server uses Stream Aggregate.

How it Works

- Reads rows one by one
- Groups consecutive rows with same key
- Produces aggregated output

Observation

- Used when input is:
 - Already sorted
 - Or very small (e.g., scalar subqueries)
 - It is often seen with Group by and Scalar Subqueries with Assert
-

3. Hash Aggregate

A Hash Aggregate performs aggregation using a hash table, grouping rows without requiring sorted input.

Example

```
SELECT l_suppkey, COUNT(*)
FROM lineitem
GROUP BY l_suppkey;
```

If input is not sorted, SQL Server uses Hash Aggregate.

How it Works

- Builds a hash table in memory
- Groups rows based on hash keys
- Produces aggregated results

Observation

- Used when:
 - Input is large
 - Data is not sorted
- Often appears in parallel execution plans

4. Assert

In Microsoft SQL Server, the Assert operator is a physical execution plan operator used to enforce conditions during query execution. It checks whether a specific condition holds true and throws an error if the condition is violated.

When is Assert Used?

The Assert operator is commonly used when SQL Server must guarantee certain constraints at runtime, such as:

- Scalar subqueries returning **at most one row**
- Enforcing uniqueness conditions
- Validating constraints during execution

Example

```
SELECT *
FROM customer
WHERE c_acctbal >
(
    SELECT AVG(c2.c_acctbal)
    FROM customer c2
);
```

- SQL Server computes the aggregate value
- Ensures that only **one row** is returned
- Uses **Assert** to validate this

Observation:

- Assert appears after stream aggregate or hash aggregate, the aggregate operators reduces the rows to one value and then the assert is used to verify the scalar condition

5. Cluster Index Seek and Cluster Index Scan

A **Clustered Index Seek** occurs when the database can directly locate specific rows using the index.

How it works

- Uses index structure (like a B-tree)
- Navigates directly to matching rows
- Reads only required data

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,    -- clustered index  
    Name VARCHAR(50),  
    Age INT  
);  
  
SELECT *  
FROM Students  
WHERE ID = 101;
```

Since ID is indexed, SQL Server performs a seek.

A **Clustered Index Scan** reads all rows in the table (or a large portion of it).

How it works

- Traverses entire clustered index
- Checks each row against condition

```
SELECT *  
FROM Students  
WHERE Age = 20;
```

If Age is not indexed, SQL Server scans the entire table.

Difference

Aspect	Seek	Scan
Access Type	Direct	Sequential

Efficiency	High	Low
Data Read	Minimal	Large
Used When	Indexed Columns	Non Indexed Columns
Performance	Optimized	Expensive

6. Joins:

Adaptive join 

Study on joins

Join Algorithms and Adaptive Join in Database Systems

1. Concepts

1.1 Why Joins are Needed

- Databases are normalized to avoid redundancy
- Data is distributed across multiple tables
- Join operation reconstructs relationships without duplication

1.2 General Strategy for Join Processing

Join algorithms follow a **divide-and-conquer approach**:

- Partition or organize data
- Match only relevant subsets

Two fundamental strategies:

- **Sorting**
- **Hashing**

These help reduce unnecessary comparisons

1.3 Binary Join vs Multi-way Join

- **Binary join:**
 - Joins two tables at a time
 - Standard approach used in query execution

1.4 Role of Query Optimizer

The query optimizer in SQL Server is responsible for:

- Choosing:
 - Join order (which inherently determines outer and inner tables)
 - Join algorithm (e.g., Nested Loop, Hash Join, Merge Join)

Key decisions:

- Smaller table is chosen as **outer (left)**
- Based on:
 - Table statistics
 - Number of pages

1.5 Cost Model

The primary cost metric is:

- **Disk I/O (most expensive operation)**

Ignored (simplified model):

- CPU cost
- Network cost
- Hashing/sorting overhead

Thus, cost is typically estimated in terms of the number of page accesses required.

1.6 Materialization Strategies

Early Materialization:

- Full tuples are copied during join
- Faster for later operators
- More memory usage

Late Materialization:

- Only record IDs (RIDs) are passed
- Actual data fetched later
- Efficient for column stores, reduces unnecessary data movement

Microsoft SQL Server is primarily a row-oriented database system, where data is stored and processed in a row-wise format. However, it also provides support for columnstore indexes, which enable column-oriented storage and execution for analytical workloads.

Our experiments in SQL Server were performed on row-store tables without columnstore indexes, so the performance was primarily influenced by join algorithms and execution strategies rather than columnar optimizations.

2. Join Algorithms

2.1 Nested Loop Join

Basic idea:

```
for each tuple r in R:
  for each tuple s in S:
    if match → output
```

Characteristics:

- Equivalent to cross join + filter
- Very expensive

Cost:

- $(M + (m \times N))$

Where:

- M = pages in S
- N = pages in S

Optimization 1: Block Nested Loop

Instead of tuple-by-tuple:

- Process **blocks (pages)** at a time
- Reduces number of I/O operations

Key idea:

- Read multiple pages of outer table
- Scan inner table once per block

Cost reduces significantly with buffering

Optimization 2: Index Nested Loop

for each tuple r in R :

lookup matching tuples in S using index

- Uses index instead of full scan
- Efficient when index exists

Important observation:

- If index is not present:
 - DBMS may create a temporary index
 - Use it for query
 - Discard afterward
-

2.2 Sort-Merge Join

Steps:

1. Sort both tables on join key
2. Merge using two pointers

while R and S :

if $R < S \rightarrow$ move R
if $R > S \rightarrow$ move S
if equal $\rightarrow$ output

Key Insights:

- Efficient when:

- Data is already sorted
 - ORDER BY is present
- Needs **backtracking** when:
 - Duplicate keys exist
- If join is on primary key:
 - No backtracking required

Cost:

- Sorting + merging cost
-

2.3 Hash Join

Idea:

- Use hashing to partition data

Steps:

Phase 1: Build

Insert tuples from R into hash table

Phase 2: Probe

For each tuple in S:

lookup in hash table

Key Observations:

- Similar to index nested loop:
 - But index is replaced by hash table
- Hash table is:
 - Built temporarily
 - Destroyed after query

When memory is insufficient:

Use **Partitioned Hash Join (GRACE Hash Join)**:

Steps:

1. Partition both tables using same hash function
2. For each partition:
 - Build hash table
 - Probe matching partition

Key insight:

- Matching keys will always fall into same partition
-

2.4 Comparison of Join Algorithms

Algorithm	Best Case	Worst Case	Notes
Nested Loop	Small tables	Very large tables	Simple but slow
Index Nested Loop	Indexed joins	No index	Efficient lookup
Sort-Merge	Sorted data	Sorting overhead	Useful for ORDER BY
Hash Join	Large unsorted data	Memory issues	Most common

2.5 Important Insight

- No single join algorithm is always optimal
 - Optimizer chooses based on:
 - Data size
 - Index presence
 - Memory
 - Sorting requirements
-

3. Adaptive Join (SQL Server Specific)

3.1 Concept

Adaptive Join is a **runtime decision-making join operator** in Microsoft SQL Server.

Instead of fixing the join algorithm beforehand:

- It **chooses dynamically during execution**

3.2 Structure

Adaptive Join has **three inputs**:

- Build input → left table
- Probe input → one option (e.g., Hash Join)
- Inner input → alternative (e.g., Nested Loop)

3.3 Working Principle

- Execution starts assuming one strategy
- Based on number of rows processed:
 - Switches algorithm

Example:

- If rows are small → Nested Loop
- If rows are large → Hash Join

3.4 Key Idea

Adaptive Join acts as:

- A **controller algorithm**
- Delegates execution to:
 - Nested Loop
 - Hash Join

3.5 Why Needed

Because:

- Optimizer estimates may be wrong
- Data distribution may vary

Adaptive Join reduces:

- Risk of bad plan selection

Index advisor

Index Advisor in MSSql Server

Index recommendations in SQL Server are generated by the query optimizer to suggest potential indexes that can improve query performance. These suggestions are based on how queries access data and are displayed as **Missing Index Recommendations** in the execution plan.

1. How SQL Server Generates Index Advice

When a query is executed, SQL Server performs the following steps:

- Analyzes query predicates such as:
 - **WHERE**
 - **JOIN**
 - **GROUP BY**
 - **ORDER BY**
- Determines the optimal access path:
 - Index Seek
 - Index Scan
- If a suitable index is not available and a better execution plan is possible:
 - SQL Server records a **Missing Index recommendation**

These recommendations are:

- Heuristic-based
- Not automatically applied
- Intended to assist developers/DBAs

Note:

- SQL Server **does not automatically create indexes** (especially in on-premise or Linux environments)
 - Recommendations must be manually reviewed and implemented
 - Suggestions are based only on the **current query workload**
-

2. Using Azure Data Studio

In Azure Data Studio:

1. Connect to the SQL Server instance
2. Open a new query window
3. **SET STATISTICS XML ON**
4. Execute the query
5. View the **execution plan tab**

If a missing index exists:

- It appears as a **green hint / recommendation**
- Includes:
 - Suggested index columns
 - Estimated improvement

Azure Data Studio:

- Only **displays recommendations**
 - Does **not automatically tune or create indexes**
-

3. When Index Advice is Generated

Index recommendations are generated **only when**:

- Queries contain explicit predicates:
 - WHERE
 - JOIN
 - GROUP BY
 - ORDER BY
 - SQL Server detects:
 - Full table scans
 - Inefficient access patterns
-

4. When Index Advice is NOT Generated

4.1. Row-Level Security (RLS)

- RLS predicates are **not considered** for index recommendations
- They are treated as **security filters**, not performance hints

Reason:

- RLS is applied **after access path selection**
- Optimizer cannot rely on user-specific filtering

4.2. Queries without Filters

Example:

```
SELECT COUNT(*) FROM table_name;
```

- Requires full table scan
- No selective predicate → no index suggestion

4.3. Policy-Based or Dynamic Filters

- Not used for index generation
 - Avoids dependency on runtime/user-specific conditions
-

5. Selection of Policies and Queries for Index Evaluation

5.1 Policy Selection

To evaluate the effectiveness of index recommendations in the presence of RLS policies, a representative set of policies was selected based on the categories defined in the paper “*Are the Database Engines Ready for Data Compliance Policies?*”.

The goal was to cover diverse types of policy semantics while ensuring that the selected policies were sufficiently complex to impact query execution.

Selected Policy Categories

The following categories were mentioned in the paper:

- Local row-level filtering policies
- Relational (join-based) filtering policies
- Existence-based (semi-join) policies
- Universal / violation-free (anti-join) policies
- Aggregation and group-level policies
- Baseline-comparative policies
- Proportionality policies
- Statistical dependence policies

From these categories, one representative policy was selected for each, including:

- P14, P15 (relational policies)
 - P19 (existence-based)
 - P39 (anti-join)
 - P40 (aggregation-based)
 - P35 (baseline-comparative)
 - P16 (proportionality)
 - P50 (statistical dependence)
-

5.2 Conversion of Policies for Index Recommendation

Since SQL Server does not generate index recommendations for RLS predicates directly, the policies were:

- Converted from **inline table-valued functions** into equivalent **standalone SQL queries** and executed independently in Azure Data Studio to obtain **missing index recommendations** from the query optimizer

Based on these recommendations:

- Index creation scripts were generated
 - Suggested indexes were manually created
 - These indexes were later used in experimental evaluation
-

5.3 Observations During Index Recommendation

- In some cases, **no missing index was suggested**, indicating either:
 - Existing indexes were sufficient, or
 - The optimizer did not find a beneficial index
- In other cases, index suggestions led to **significant improvements in execution time**

Examples include:

- P14: Improvement observed after indexing (C_NationKey, C_AcctBal)
- P19: Improvement after indexing (PS_SuppKey)
- P40: Improvement after indexing (L_SuppKey)

However:

- Some policies (highly nested correlated subqueries) did not yield usable results

- In particular, experiments involving policy 40 could not be completed due to complexity involved due to its correlated nested structure.
-

5.4 Query Selection

To evaluate the impact of indexes in combination with RLS policies, a subset of TPC-H queries was selected.

These queries were chosen because they:

- Involve multiple tables
- Interact with selected policies
- Represent complex analytical workloads

Selected Queries and Policy Combinations

- **Query 21** → tables: lineitem, supplier, orders
 - Policies: (P16, P19), (P40, P19), (P35, P19)
 - **Query 7** → tables: supplier, lineitem, orders, customer
 - Policies: combinations of (P19, P16, P39), (P19, P40, P39), (P19, P35, P39), etc.
 - **Query 8** → tables: customer, part, supplier, lineitem, orders
 - Policies: combinations involving (P39, P19, P16), (P14, P19, P40), etc.
 - **Query 12** → tables: orders, lineitem
 - Policies: (P16), (P40)
-

5.5 Experimental Configurations

Experiments were conducted under multiple indexing scenarios to understand the role of indexes in both queries and policies.

For each query-policy combination, the following configurations were evaluated:

1. **No indexes created**
 - Neither query-specific nor policy-specific indexes
 - Observed that:
 - SQL Server sometimes used **index spool operators** for policies
 - Missing indexes were suggested for queries
2. **Indexes for queries only**
 - Only indexes suggested for query predicates were created
3. **Indexes for policies only**
 - Only indexes derived from policy predicates were created

4. Indexes for both queries and policies

- Combined indexing strategy
-

5.6 Results

1. Local Row-level Filtering Policies
2. Relational (Join-based) Filtering Policies → **P14 (customer) P15(customer)**
3. Existence-based (Semi-join) Policies → **P19 (part)**
4. Universal / Violation-free (Anti-join) Policies → **P39 (customer)**
5. Aggregation and Group-level Policies → **P40 (lineitem)**
6. Baseline-comparative Policies → **P35 (lineitem)**
7. Proportionality Policies → **P16 (lineitem)**
8. Statistical Dependence Policies → **P50 (orders)**

- 1. **P48 (lineitem)** Missing index not suggested : Time - 00:01:35.060
- 2. **P24 (supplier)** → Missing index not suggested : Time - 00:00:04.506

**SET 1 → policies as queries → without index (time before) and with index (time after).
Index as suggested by the index advisor has been created**

1. For all policies in this category no missing index was suggested
2. P14 → Time before - 00:00:00.316 Time after - 00:00:00.189
 Idx → ([C_NationKey],[C_AcctBal])
 P15 → Time before - 00:00:00.399 Time after - 00:00:00.119
 Idx → ([O_OrderDate])
3. P19 → Time before - 00:00:02.659 Time after - 00:00:00.377
 Idx → ([PS_SuppKey])
4. P39 → Time before - 00:00:12.199 Time after - 00:00:11.616
 Idx → ([L_ShipMode])
5. P40 → Time before - 00:00:12.530 Time after - 00:00:09.236
 Idx → ([L_SuppKey])
6. P35 → Time before - 00:00:10.977 Time after - 00:00:10.786
7. P16 → Time before - 00:00:11.430 Time after - 00:00:10.868
 Idx → ([O_CustKey])

8. No missing idx was suggested

q21

(P16, P19)

Without policy idx → 00:00:02.740

With policy idx → 00:00:02.794

With policy + query → 00:00:02.724

With query idx → 00:00:02.709

	Without policy idx or query idx	With policy idx only	With policy + query idxs both	With query idx only
q8-(P39, P19, P16)	CPU time = 15085 ms, elapsed time = 14123 ms. Total execution time: 00:00:15.012	CPU time = 2280 ms, elapsed time = 252 ms. Total execution time: 00:00:01.003	CPU time = 2029 ms, elapsed time = 229 ms. Total execution time: 00:00:00.777	CPU time = 13305 ms, elapsed time = 12503 ms. Total execution time: 00:00:12.967
q8-(P39, P19, P40)				
q8-(P39, P19, P35)	CPU time = 792654 ms, elapsed time = 210261 ms. Total execution time: 00:03:30.797	CPU time = 1162299 ms, elapsed time = 181413 ms. Total execution time: 00:03:01.966	CPU time = 1261055 ms, elapsed time = 179733 ms. Total execution time: 00:03:00.340	CPU time = 613229 ms, elapsed time = 261237 ms. Total execution time: 00:04:21.794
q8-(P14, P19, P16)	CPU time = 15555 ms, elapsed time = 14271 ms. Total execution time:00:00:15.024	CPU time = 299 ms, elapsed time = 102 ms. Total execution time: 00:00:00.619	CPU time = 286 ms, elapsed time = 97 ms. Total execution time: 00:00:00.702	CPU time = 460 ms, elapsed time = 92 ms. Total execution time: 00:00:00.706
q8-(P14, P19, P40)				
q8-(P14, P19, P35)	CPU time = 633 ms, elapsed time = 108 ms. Total execution time: 00:00:00.602	CPU time = 285 ms, elapsed time = 99 ms. Total execution time: 00:00:00.627	CPU time = 375 ms, elapsed time = 119 ms. Total execution time: 00:00:00.691	CPU time = 557 ms, elapsed time = 102 ms. Total execution time: 00:00:00.591
q12-(P16)	CPU time = 32434 ms, elapsed time = 28732 ms. Total execution time:00:00:28.916	CPU time = 31032 ms, elapsed time = 26844 ms. Total execution time: 00:00:27.015	CPU time = 30900 ms, elapsed time = 28248 ms. Total execution time: 00:00:28.485	CPU time = 29834 ms, elapsed time = 27168 ms. Total execution time: 00:00:27.423

q12-(P40)				
-----------	--	--	--	--

PLOTS TO BE DONE

PCI

Policy Coverage Index

The purpose of introducing the Policy Coverage Index (PCI) is to evaluate the performance impact of indexing in the presence of Row-Level Security (RLS) policies in Microsoft SQL Server.

RLS policies often involve multiple joins across tables (such as supplier, nation, region, partsupp, etc.), which can significantly degrade performance if appropriate indexes are not present. Therefore, PCI is used to ensure that all columns involved in policy predicates are efficiently indexed.

A **Policy Coverage Index (PCI)** refers to: *Indexes that are created specifically to support the columns used in a given RLS policy.*

- Each policy has its own set of indexes
- These indexes are not global, but policy-specific
- PCI ensures that all relevant columns used in:
 - WHERE conditions
 - JOIN conditions are indexed

Indexing Strategy

The experiment follows a strict indexing strategy:

1. Retain only essential indexes
 - Primary Key (PK)
 - Foreign Key (FK)
2. Remove all other indexes
 - To avoid interference from pre-existing optimizations
 - To ensure consistent and controlled results
 - [to drop all non pk fk](#) → this script was used to executed to remove the other indexes
3. Create PCI indexes
 - For each policy separately
 - Based only on columns used in that policy

Why Only Single-Column Indexes?

Only single-column indexes are used for PCI due to:

- Easier analysis of individual column impact
- Avoiding complexity of composite index interactions

Composite indexes may improve performance but make it difficult to isolate the contribution of each column.

PCI Creation Approach

For each policy:

- Identify all columns used in Predicate conditions and Joins
- Create single-column indexes:

```
CREATE NONCLUSTERED INDEX idx_<table>_<column>  
ON <table>(<column>);
```

- Store these scripts in: pci_indexes folder

Query 18 n 19 seek scan

1. Objective

The objective of this experiment is to analyze the behavior of the query optimizer in Microsoft SQL Server with respect to:

- Clustered Index Seek vs Clustered Index Scan
- Impact of:
 - Index structure (composite indexes and leading columns)
 - Row-Level Security (RLS) predicate functions
- Understanding why:
 - Similar keys may lead to different access methods (seek vs scan)
 - Different keys may lead to expected behavior based on index ordering

2. Experiment

2.1 Setup

- Database: TPCCH schema
- Queries executed:

```
SELECT COUNT(*) FROM <table>;
```

- Row-Level Security policies implemented using predicate functions:
 - [P18](#) → Supplier filtering
 - [P19](#) → Part filtering
 - [P21](#) → Orders filtering
 - [P23](#) → Customer filtering

2.2 Predicate Logic

P18: Supplier visibility

- Filters suppliers based on parts with specific name
- Uses: **ps_supkey**

P19: Part visibility

- Filters parts supplied from Europe
- Uses: **ps_partkey**

P21: Order visibility

- Filters orders based on total revenue
- Uses aggregation on **lineitem** grouped by **I_orderkey**

P23: Customer visibility

- Filters customers based on number of suppliers
- Uses join between **orders** and **lineitem** on **l_orderkey**

2.3 Index Configurations

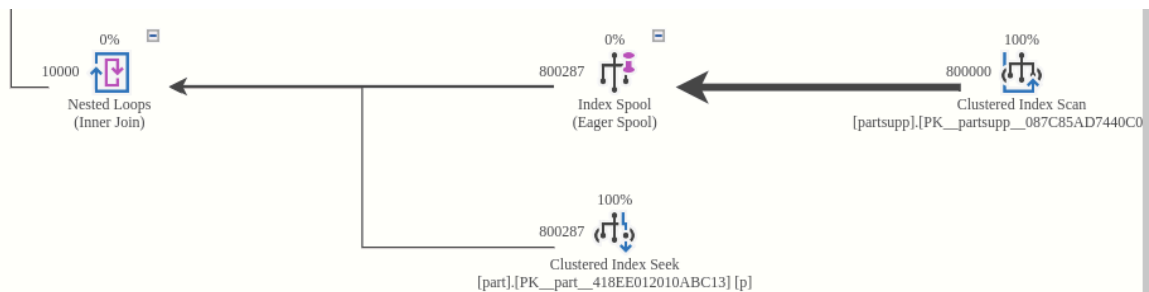
For P18 & P19:

- Composite indexes tested:
 - (ps_partkey, ps_suppkey)
 - (ps_suppkey, ps_partkey)

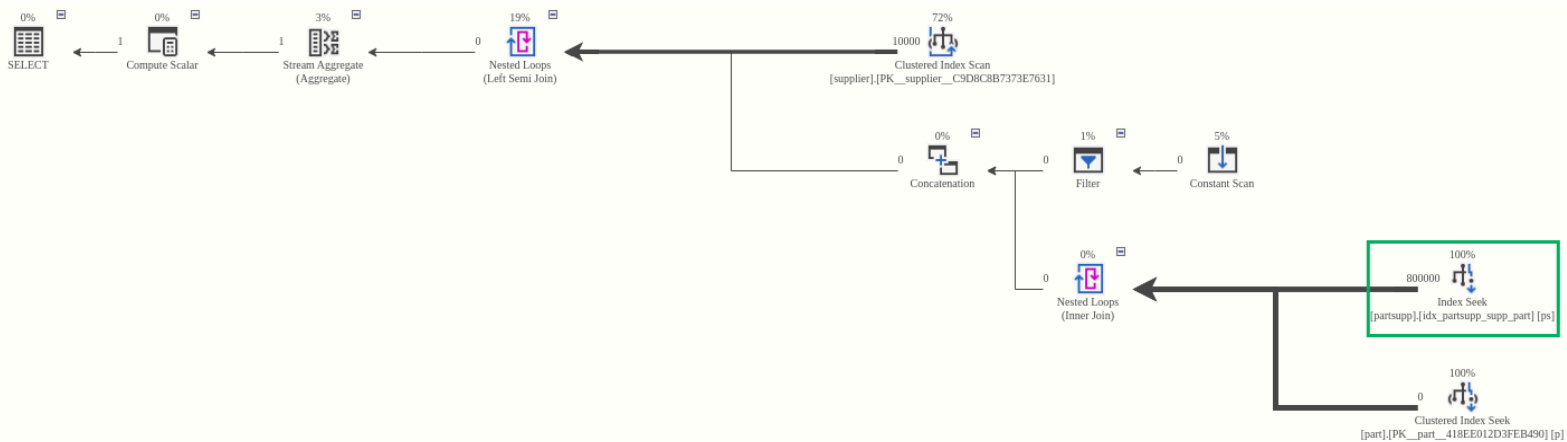
3. Observations

P18 and P19

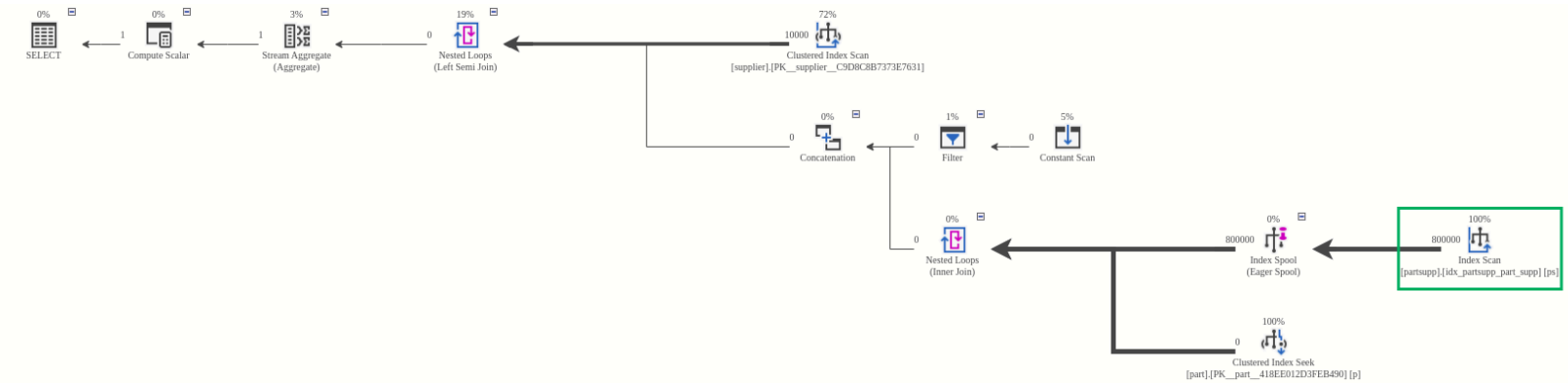
P18 query plan (without indexing)



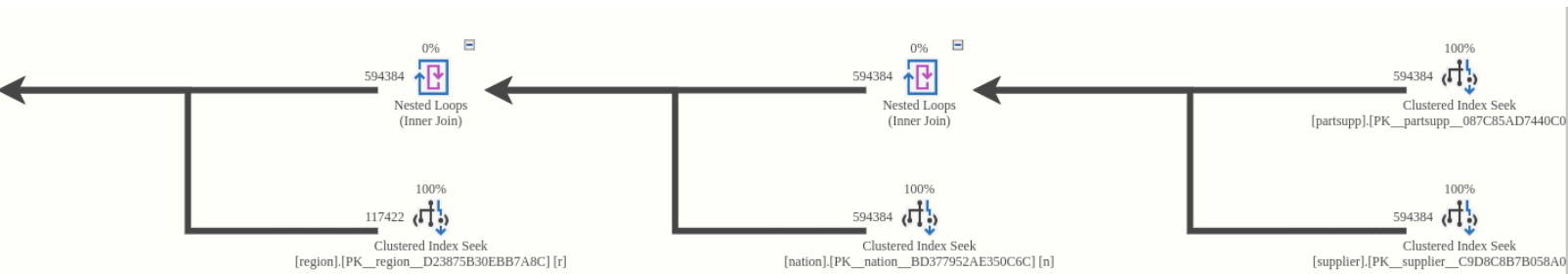
P18 with supp_part composite index



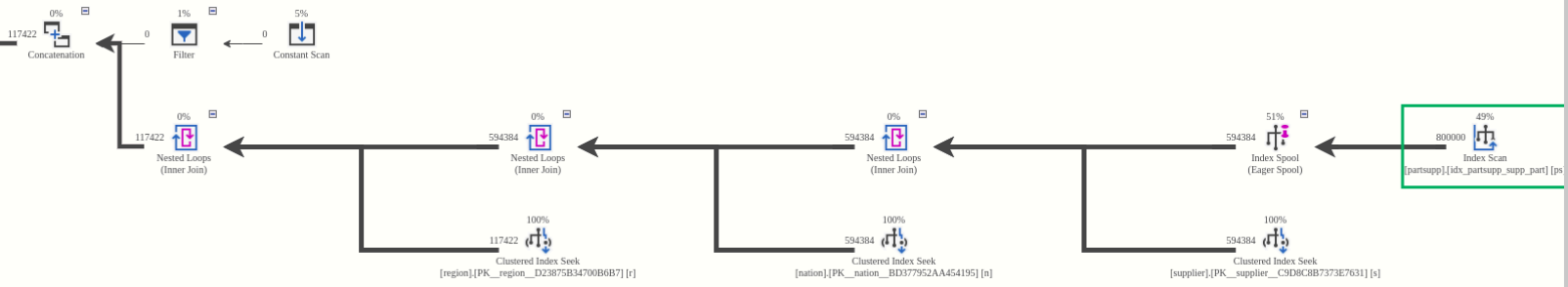
P18 with part_supp composite index



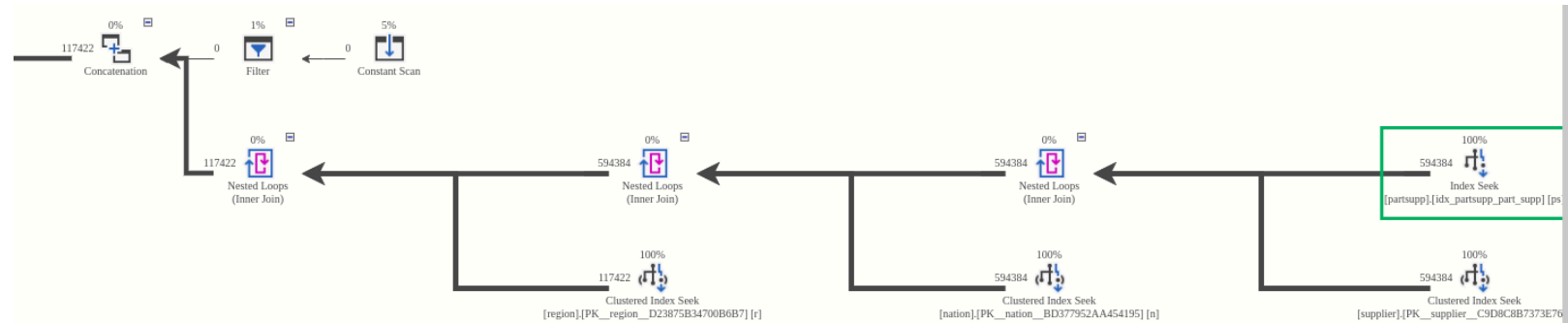
P19 without indexing



P19 with supp_part composite index



P19 with part_supp composite index



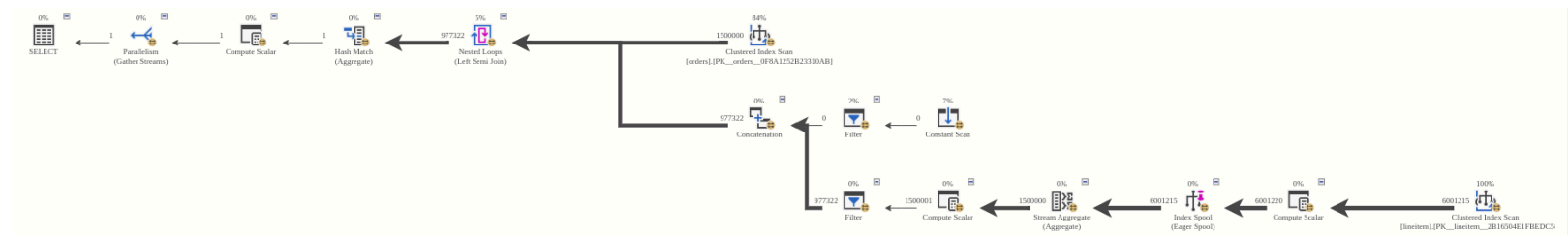
Policy	Key Used	Index Leading Column	Operation
P18	ps_suppkey	(ps_suppkey, ps_partkey)	Seek
P18	ps_suppkey	(ps_partkey, ps_suppkey)	Scan
P19	ps_partkey	(ps_partkey, ps_suppkey)	Seek
P19	ps_partkey	(ps_suppkey, ps_partkey)	Scan

- SQL Server uses left-most prefix rule
- If filter condition matches leading column → SEEK
- Otherwise → SCAN

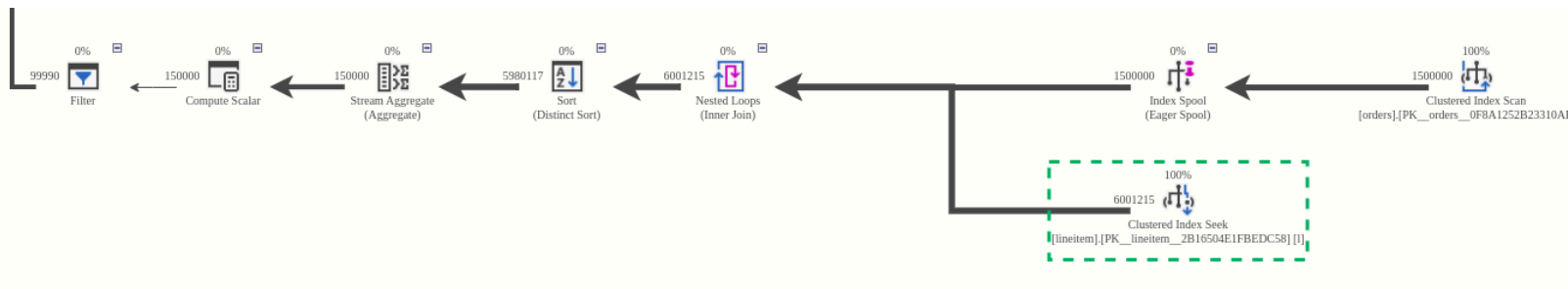
The behaviour (seek and scan) is according to the leading columns in the index created for P18 and P19. Without creation of the composite index manually as well it was showing similar behaviour according to the leading column (ps_partkey) as in the P18 and P19

P21 and P23

P21 without indexing



P23 without indexing



Policy	Table	Key	Operation
P21	lineitem	l_orderkey	Scan
P23	lineitem	l_orderkey	Seek

Even though both use same key (l_orderkey), behavior differs due to:

P21:

```
SELECT SUM(l_extendedprice * (1 - l_discount))
FROM lineitem
WHERE l_orderkey = @orderkey
```

In policy P21, the query involves an aggregation using SUM, which requires processing all rows that match a given l_orderkey. Since each order can have many corresponding rows in the lineitem table, the number of matching rows is relatively large, resulting in low selectivity. In such cases, even if an index exists, the optimizer may determine that scanning the data is more efficient than performing multiple index seeks. This is because a scan allows sequential access to all relevant rows with lower overall I/O cost, making it a cheaper option.

P23:

```
SELECT COUNT(DISTINCT l_suppkey)
FROM orders o JOIN lineitem l
ON l.l_orderkey = o.o_orderkey
```

In P23, the query runs inside a correlated subquery and is usually executed using a nested loop. For each outer row (customer or order), SQL Server looks up matching rows in the lineitem table using l_orderkey. Since only a small number of rows are accessed each time (high selectivity), the optimizer prefers an index seek, as it is faster for targeted lookups.

Cost-Based Optimization

SQL Server chooses:

- Scan → when reading large portion is cheaper
- Seek → when few rows are needed

So behavior is not only index-based, but also Query shape, Estimated rows, Cost model.

Conclusion

P18 & P19

- Behavior depends strictly on **leading column of index**
- Confirms standard indexing rules

P21 vs P23

- Same key does **NOT guarantee same access method**
- Query structure plays a critical role

long query + parallelism

TPC-H Query Performance Analysis and Execution Control in Microsoft SQL Server

1. Introduction

This section documents a series of experiments conducted on TPC-H queries executed in Microsoft SQL Server, with a particular focus on queries that were run with Policy 40 enabled. These queries exhibited severe performance issues, including extremely long execution times and system instability. The objective of this work was to understand the root causes of these issues, analyze query execution behavior using system tools, and explore methods to control and mitigate long-running queries.

2. Fundamental Concepts

2.1 Dynamic Management Views (DMVs)

DMVs are system views that provide **real-time insights into SQL Server internals**.

They help in:

- Monitoring active queries
- Identifying slow queries
- Detecting blocking and waits
- Inspecting execution details

Why DMVs were used:

- To identify long-running queries
 - To understand why queries are slow
 - To safely terminate problematic queries
-

2.2 Query Execution States

A query in SQL Server can be in one of the following states:

- **Running**
 - Actively executing on CPU
- **Runnable**
 - Ready to execute but waiting for CPU

- **Suspended**
 - Waiting for a resource:
 - Disk I/O
 - Locks
 - Memory
 - Parallel thread synchronization
-

2.3 Wait Types and Their Significance

Wait types indicate the reason why a query is not progressing. They provide insight into the bottlenecks affecting query performance.

Lock-related waits:

- LCK_M_X → Exclusive lock wait
 - Occurs when a query is waiting to acquire an **exclusive lock** in order to modify data.
 - An exclusive lock blocks both:
 - Other reads (shared locks)
 - Other writes (exclusive/update locks)
- LCK_M_S → Shared lock wait
 - Occurs when a query is waiting to acquire a **shared lock** to read data.
 - Shared locks allow multiple readers but block writers.
- LCK_M_U → Update lock wait
 - Occurs when a query is waiting to acquire an update lock, which is used during a read operation that may later turn into a write.

Difference between Update Lock (LCK_M_U) and Exclusive Lock (LCK_M_X)

Both update locks and exclusive locks are related to data modification, they serve different purposes and occur at different stages of query execution.

1. Purpose

- **Update Lock (U)**
 - Used when SQL Server **intends to update a row but has not yet modified it.**
 - It is taken during the **read phase before the actual update.**
- **Exclusive Lock (X)**
 - Used when SQL Server is **actually modifying the data.**

2. Execution Flow

A typical update operation internally follows this sequence:

1. Acquire **Update lock (U)** while scanning and identifying rows
2. Once the row is confirmed for modification, convert it to
3. **Exclusive lock (X)** to perform the update

3. Why Update Locks Exist (Deadlock Prevention)

Update locks are primarily used to **prevent deadlocks**.

Without update locks:

- Two transactions could both acquire shared locks while reading
- Then both try to upgrade to exclusive locks
- This leads to a deadlock (both waiting on each other)

With update locks:

- Only one transaction can hold an update lock on a resource at a time
- Other transactions must wait earlier
- This prevents the deadlock situation

Resource waits:

- PAGEIOLATCH → Waiting for disk read
- WRITELOG → Waiting for log write
- RESOURCE_SEMAPHORE → Waiting for memory

Parallelism waits:

- CXCONSUMER
 - Parent thread waiting for child threads
 - Occurs in parallel execution
- CXSYNC_PORT
 - Synchronization between parallel threads
 - Data exchange between producer and consumer threads

2.4 Blocking vs Internal Parallelism

A key distinction in performance analysis is between blocking and internal waits caused by parallel execution.

Blocking occurs when one query prevents another from proceeding due to lock contention. This can be identified when the `blocking_session_id` is different from the `session_id`.

In contrast, when the `blocking_session_id` is equal to the `session_id`, the wait is internal to the query. This situation typically arises during parallel execution, where multiple threads of the same query coordinate with each other. In such cases, there is no external blocking, and the delay is due to internal synchronization.

Conclusion:

- Not all waits indicate blocking
 - Parallel queries can appear “blocked” but are not
-

2.5 Parallelism and Degree of Parallelism (MAXDOP)

SQL Server can execute queries using multiple threads to improve performance. This behavior is controlled by the Degree of Parallelism (MAXDOP). Parallel execution divides the workload across multiple threads, but it also introduces overhead due to synchronization and communication between threads.

MAXDOP can be controlled at two levels. At the query level, it can be specified using a query hint. At the server level, it can be configured globally using system procedures. Reducing MAXDOP forces queries to execute using fewer threads, which can help in diagnosing performance issues related to parallelism.

Control options:

- Query level → `OPTION (MAXDOP 1)`
 - Server level → `sp_configure` ([num_threads_config.sql](#))
-

3. Experiment 1: Identification and Termination of Long-Running Queries

3.1 Objective

The objective of this experiment was to identify long-running queries, analyze their execution behavior, and terminate queries that exceeded a specified execution time threshold. This was particularly important for queries enabling Policy 40, which caused significant performance degradation due to its nested correlated structure.

3.2 Experiment

The experiment was conducted using three DMVs to systematically analyze query execution.

The first query ([dmv1.sql](#)) was used to identify currently running queries. It retrieves information such as session ID, execution status, wait type, total execution time, and the SQL text of the query. By ordering results based on execution time, it highlights the slowest queries.

The second query ([dmv2.sql](#)) was used to detect blocking. It identifies relationships between blocking and blocked sessions and provides information about the wait type and duration.

The third query ([dmv3.sql](#)) was used to verify query details before termination. It lists all active sessions and their associated SQL text, ensuring that the correct query is targeted for termination.

Once a problematic query was identified, it was terminated using the KILL command.

```
KILL <SPID>;
```

3.3 Observations

Blocking analysis:

The analysis revealed that there was no actual blocking in the system. The `blocking_session_id` was equal to the `session_id`, indicating that the observed delays were due to internal query execution rather than lock contention.

Parallelism observations:

Parallelism-related wait types such as `CXCONSUMER` and `CXSYNC_PORT` were observed. These wait types indicated that the query was executing in parallel and that threads were waiting on each other for data exchange and synchronization.

System behavior:

The queries consumed significant system resources and caused the system to become unresponsive. This confirmed that the issue was related to query execution complexity rather than external blocking.

4. Experiment 2: Controlling Parallelism

4.1 Objective

The objective of this experiment was to determine whether parallel execution was contributing to the observed performance issues and to evaluate the effect of limiting the number of threads.

4.2 Experiment

Parallelism can be controlled at both the query level and the server level. At the query level, the `MAXDOP` hint was used to restrict execution to a single thread.

Query-level control:

OPTION (MAXDOP 1)

Global configuration:

```
EXEC sp_configure 'show advanced options', 1;
```

```
RECONFIGURE;
```

```
EXEC sp_configure 'max degree of parallelism', 1;
```

```
RECONFIGURE;
```

These changes allowed for a comparison between parallel and non-parallel execution behavior.

4.3 Observations

When **parallelism was enabled**, queries were observed to remain in the **suspended state** for extended periods. This indicated that threads were waiting for synchronization or resource availability.

When **parallelism was disabled**, queries transitioned to the **runnable state** but still **did not execute** efficiently. This suggested that the issue was not solely due to parallelism but also related to other factors such as **query complexity** and resource contention.

The results indicated that while parallelism introduced overhead, it was not the primary cause of the performance issues.

5. Analysis of Policy 40 Query Behavior

5.1 Nature of the Query

The Policy 40 involved nested correlated subqueries. In such queries, the inner query is executed repeatedly for each row of the outer query.

This leads to a significant increase in computational cost, especially when operating on large datasets such as those in TPC-H.

5.2 Impact on Performance

The repeated execution of the inner query resulted in extremely high execution times. The complexity of the query increased with the size of the dataset, leading to exponential growth in processing time.

This behavior explains why the queries caused system instability and required manual intervention.

5.3 Conclusion

The root cause of the performance issues was the inherent complexity of the query structure combined with large data volumes. While techniques such as query termination and parallelism control helped manage the symptoms, they did not address the underlying problem. So the flattening was tried as a part of the next experiment.

Query Flattening

1. Flattening using CTE

When a correlated nested RLS policy is written using a CTE, SQL Server attempts to inline the CTE into the predicate function and the outer query. However, the effectiveness of flattening depends on whether the correlation can be transformed into a set-based expression. If the correlation is expressed in a way that can be de-correlated, the optimizer integrates the CTE logic into the overall query tree.

1.1 Behavior in RLS

- The correlated logic inside the CTE is analyzed along with the outer query
- If de-correlation is possible, the predicate is merged into a unified query tree
- The policy condition is evaluated together with the user query
- No separate materialization of the CTE is introduced

1.2 Optimization Impact

Because the CTE can be flattened in many cases:

- Correlated predicates may be transformed into joins
- Filters from the outer query can be pushed into the predicate logic
- Join reordering across the policy and main query becomes possible
- Redundant evaluations of the correlated logic may be reduced

This leads to a **set-based execution plan**, provided the correlation is successfully removed during optimization.

1.3 Implication

Using a CTE in correlated RLS policies allows the optimizer an opportunity to de-correlate and flatten the logic. When successful, this avoids repeated execution of the predicate and results in efficient query execution. If de-correlation is not possible, the benefits of flattening may be limited.

2. Flattening using VIEW

When a correlated nested RLS policy references a view, SQL Server expands the view definition into the predicate function during optimization. The ability to flatten depends on whether the correlation between the outer query and the view can be simplified into a set-based form.

2.1 Behavior in RLS

- The view definition is expanded into the predicate function
- Correlated conditions are combined with the outer query
- The optimizer attempts to merge the policy and query into a single execution tree
- De-correlation is attempted across the view boundary

2.2 Optimization Impact

If flattening is successful:

- Correlated subqueries may be rewritten as joins
- Predicate pushdown can occur across the view definition
- Join reordering and expression simplification are enabled
- Repeated evaluation of correlated logic can be minimized

However, if the correlation remains complex, full flattening may not be achieved, limiting optimization.

2.3 Implication

Views provide abstraction for correlated policy logic, but their performance depends on the optimizer's ability to de-correlate the query. When flattening succeeds, the policy is executed efficiently as part of a set-based plan. Otherwise, residual correlation may introduce additional execution costs.

3. Behavior of CROSS APPLY in Correlated Policies

CROSS APPLY directly represents a correlated execution pattern in SQL Server. In the context of correlated nested RLS policies, it enforces row-by-row evaluation and cannot be flattened into a single set-based query.

3.1 Behavior in RLS

- The predicate logic inside CROSS APPLY is executed for each row of the outer query
- The correlation between outer and inner queries is explicitly maintained
- The policy is not merged into a unified query tree
- The inner logic is repeatedly evaluated

3.2 Optimization Impact

Since CROSS APPLY is not flattened:

- De-correlation is not performed
- Predicate pushdown is limited
- Join reordering across the APPLY boundary is restricted
- Execution follows a nested loop strategy
- The inner query is executed multiple times

This results in a **row-by-row execution model**, which can significantly increase cost.

3.3 Implication

In correlated RLS policies, CROSS APPLY makes the correlation explicit and prevents flattening. While it provides flexibility in expressing row-dependent logic, it leads to repeated evaluation and higher execution cost. Compared to CTEs and views, it offers less opportunity for global optimization.

Flattening of P40

1. Original Query

In the original implementation of P40, the policy computes **import and export values inside correlated subqueries**. These computations are executed repeatedly for each row of the outer query.

```
SELECT ...
FROM supplier s
WHERE (
    SELECT SUM(...)
    FROM lineitem l
    WHERE l.supkey = s.supkey AND ... -- export
) >
(
    SELECT SUM(...)
    FROM lineitem l
    WHERE l.supkey = s.supkey AND ... -- import
)
AND nation = 'GERMANY';
```

- Import and export are computed **inside nested subqueries**
- The computation is **repeated for each supplier row**
- Leads to:
 - Correlated execution
 - Nested loop behavior
 - High execution cost

2. Flattened using CTE

The correlated logic is rewritten using CTEs by **separating import and export computations into independent result sets**.

```
WITH Export AS (  
    SELECT l_suppkey, SUM(...) AS export_val  
    FROM lineitem  
    GROUP BY l_suppkey  
)  
,  
Import AS (  
    SELECT l_suppkey, SUM(...) AS import_val  
    FROM lineitem  
    GROUP BY l_suppkey  
)  
SELECT ...  
FROM supplier s  
JOIN Export e ON s.suppkey = e.l_suppkey  
JOIN Import i ON s.suppkey = i.l_suppkey  
WHERE e.export_val > i.import_val  
AND nation = 'GERMANY';
```

- Import and export are computed **once globally**, not per row
- Correlated subqueries are replaced with **joins**
- The query becomes **set-based**

The execution of P40 after flattening with CTE also took a lot of time to run as the intermediate were large (didn't generate output)

3. Flattened using VIEW / Precomputation

Instead of computing import and export within the query, they are **precomputed and stored** as reusable objects.

```
-- Precomputed view  
CREATE VIEW Export_View AS  
SELECT l_suppkey, SUM(...) AS export_val  
FROM lineitem  
GROUP BY l_suppkey;  
  
CREATE VIEW Import_View AS  
SELECT l_suppkey, SUM(...) AS import_val  
FROM lineitem  
GROUP BY l_suppkey;
```

```
SELECT ...  
FROM supplier s  
JOIN Export_View e ON s.suppkey = e.l_suppkey  
JOIN Import_View i ON s.suppkey = i.l_suppkey  
WHERE e.export_val > i.import_val  
AND nation = 'GERMANY';
```

- Computation moved **outside the policy/query**
- Import/export reused instead of recomputed
- Enables:
 - Indexing on computed results
 - Reduced execution overhead

The execution time was less when the policy was rewritten with views.

4. Flattened using CROSS APPLY

The nested subqueries are rewritten using CROSS APPLY to attach computed values per row.

```
SELECT ...
FROM supplier s
CROSS APPLY (
  SELECT SUM(...) AS export_val
  FROM lineitem l
  WHERE l.supkey = s.supkey
) e
CROSS APPLY (
  SELECT SUM(...) AS import_val
  FROM lineitem l
  WHERE l.supkey = s.supkey
) i
WHERE e.export_val > i.import_val
AND nation = 'GERMANY';
```

- Nested queries replaced with APPLY operators
- Computation still depends on each row

The flattening of P40 demonstrates that:

- Optimization is achieved when:
 - Correlated subqueries are converted into **set-based operations**
- CTE improves structure but may not eliminate cost
- VIEW-based approach avoids recomputation entirely
- CROSS APPLY restructures the query but does not remove correlation
- The time complexity is less in case of the policy flattened using VIEW and CROSS APPLY

Thus, flattening is most effective when it transforms **row-wise repeated computation into reusable set-based results**.

UDF

User Defined Functions (UDFs)

1.Introduction

A **User Defined Function (UDF)** in Microsoft SQL Server is a programmable database object that encapsulates reusable logic and can be invoked within SQL queries. UDFs enable modularization of complex conditions and improve maintainability by separating logic from query structure.

UDFs are broadly classified into three types:

- **Scalar UDFs**: Return a single scalar value
- **Inline Table-Valued Functions (TVFs)**: Return a table expression defined by a single query
- **Multi-statement TVFs**: Return a table constructed through procedural steps

In this work, **scalar UDFs** are used, as they naturally map to boolean predicates required for filtering rows.

2. Scalar UDF Execution Model

A scalar UDF is invoked for each row in which it appears. Conceptually, its execution follows:

For each input row:

 evaluate function(parameters)
 return scalar value

This row-wise execution contrasts with set-based operations that SQL Server typically optimizes for.

3. UDF Inlining in SQL Server

Recent versions of SQL Server introduce an optimization called **scalar UDF inlining**, where the optimizer attempts to transform a scalar UDF into an equivalent relational expression. When inlining occurs:

- The function body is expanded into the main query
- Execution becomes set-based rather than row-by-row
- The optimizer can reorder joins and push predicates

However, inlining changes the execution semantics and makes the UDF no longer behave as an independent unit.

4. Black-Box Execution

A UDF is said to behave as a **black box** when:

- Its internal logic is not visible to the query optimizer
- It is executed independently for each row
- No transformations (e.g., predicate pushdown or join reordering) are applied across its boundary

Black-box execution is important when studying:

- Isolation of logic
 - Absence of optimizer-driven rewrites
 - Effects on query performance and evaluation order
-

5. Policy Enforcement Using UDFs

Row-level policies in SQL Server are typically implemented using **Row-Level Security (RLS)**, where predicates are attached to tables and enforced automatically. In contrast, this approach explores:

Implementing policy logic using scalar UDFs and applying it explicitly within query predicates.

5.1 Conceptual Difference from RLS

Aspect	RLS	UDF-based enforcement
Application	Automatic	Manual (via WHERE clause)
Integration	Attached to table	Embedded in query
Optimization visibility	Fully visible	Can be hidden (black box)

5.2 Motivation

The use of UDF-based policies allows controlled experimentation with:

- Execution behavior when policy logic is not exposed to the optimizer
 - The absence of cross-query or cross-table transformations
 - The effect of isolating policy evaluation from query planning
-

6. Implementation

6.1 UDF Definition

```
-- Dummy policy: show lineitem row only if supplier exists for that supkey
DROP FUNCTION IF EXISTS dbo.udf_dummy_blackbox
GO

CREATE FUNCTION dbo.udf_dummy_blackbox
(
    @supkey BIGINT
)
RETURNS INT
AS
BEGIN
    DECLARE @allowed INT = 0;

    IF EXISTS (
        SELECT 1
        FROM dbo.supplier s
        WHERE s.s_supkey = @supkey
        AND s.S_NationKey < 3
    )
    BEGIN
        SET @allowed = 1;
    END

    RETURN @allowed;
END
GO
```

6.2 Design Characteristics

Scalar Return Type

The function returns an integer (0 or 1), representing a boolean condition:

- 1 → row satisfies policy

- 0 → row does not satisfy policy

Predicate Logic

The function checks whether a supplier satisfies a condition:

A row is allowed if there exists a supplier with:
matching supplier key AND nation key < 3

6.3 Query Integration

The policy is applied explicitly in the query:

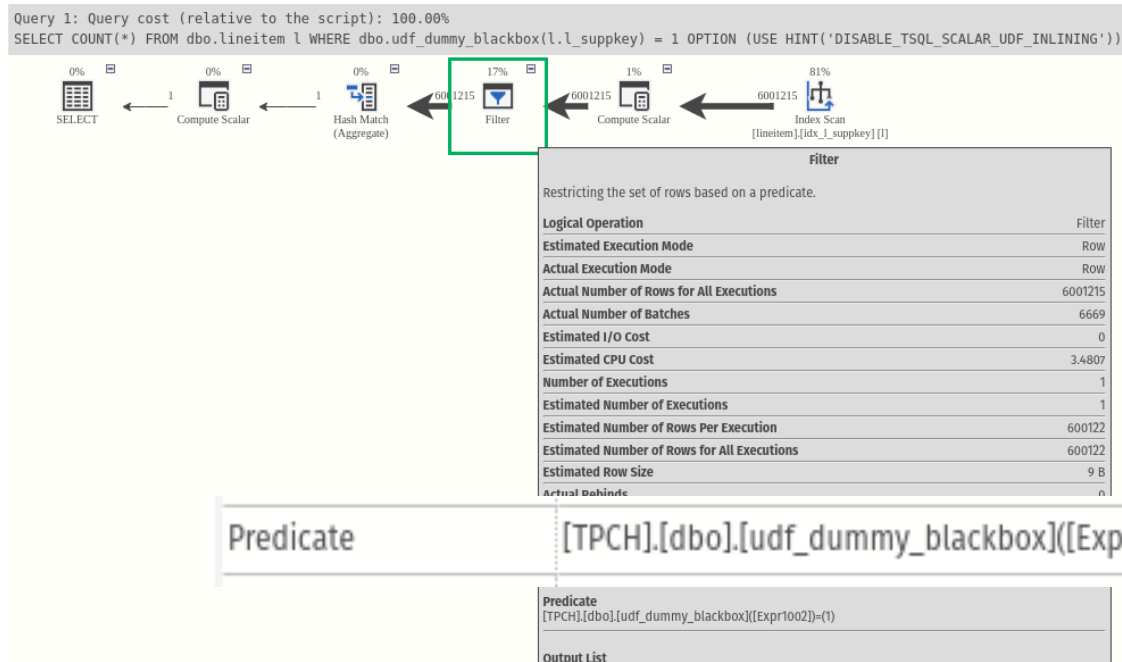
```
SELECT COUNT(*)
FROM dbo.lineitem l
WHERE dbo.udf_dummy_blackbox(l.l_suppkey) = 1
OPTION (USE HINT('DISABLE_TSQL_SCALAR_UDF_INLINING'));
```

Key Aspects

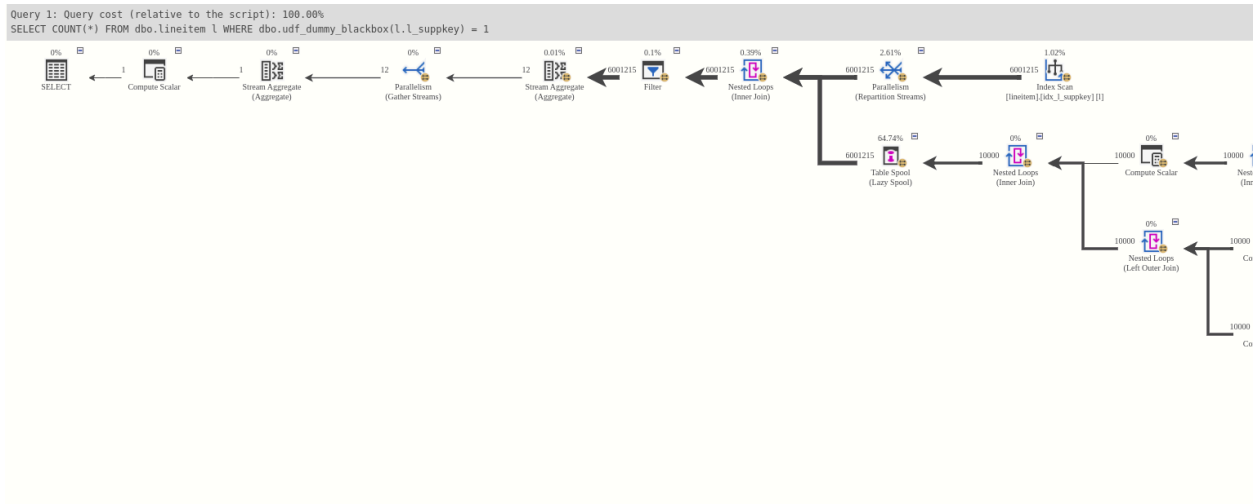
1. **Explicit invocation**
The function is called for each row of lineitem.
2. **Predicate enforcement**
Only rows for which the function returns 1 are retained.
3. **Query hint usage**

OPTION (USE HINT('DISABLE_TSQL_SCALAR_UDF_INLINING'))

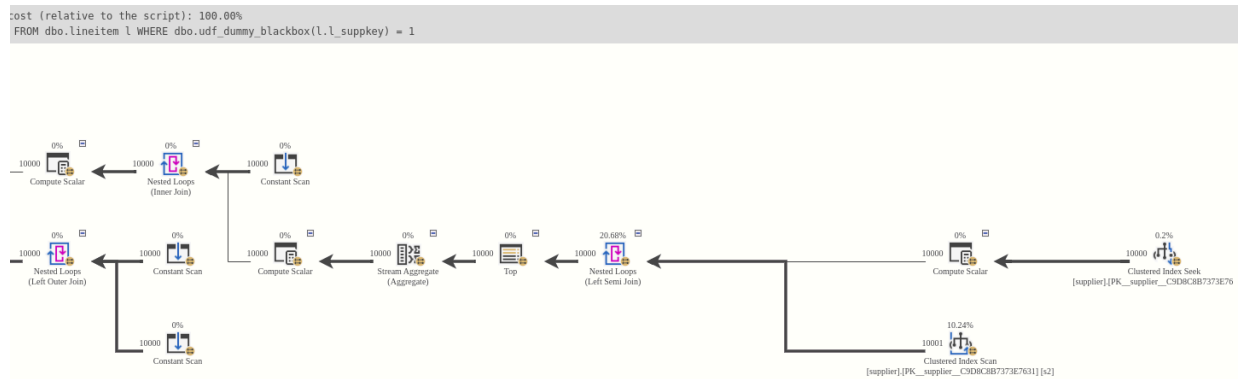
THE QUERY PLAN WITH DISABLING OF INLINING USING OPTION



THE QUERY PLAN WITHOUT DISABLING OF INLINING



continued..



This ensures:

- The optimizer does not inline the UDF
- The function remains a black box even if eligible for inlining

6.4. Execution Characteristics

When used in a query:

For each row in lineitem:

invoke `udf_dummy_blackbox(l_suppkey)`

evaluate EXISTS condition
return 0 or 1

This leads to:

- **Row-by-row evaluation**
 - No reuse of intermediate results
 - No sharing of computation across rows
-

7. Recursion in UDF-based Policy Enforcement

In a UDF-based approach, even if there are mutual dependencies (such as a function on *table1* querying *table2* and another function on *table2* querying *table1*) recursion does not occur.

This is because each UDF call is executed **independently for each row** and is treated as a **black box** by the system. The optimizer does not expand or link function calls across tables, and no recursive dependency chain is formed during execution.

As a result, the evaluation consists of separate function invocations rather than recursive calls, and mutual references between functions do not lead to recursion.

UDF experiments

Performance Evaluation of UDF, RLS, and View-based Policy Enforcement

1. Experimental Evaluation

This section presents the experimental evaluation of policy enforcement using three approaches: **UDF**, **RLS**, and **views**. The experiments are designed to study differences in **planning (parse and compile) time** and **execution time**, particularly in the presence of logical compositions such as **tautologies and contradictions**, as well as cyclic policy scenarios.

The planning time is approximated using:

```
SET STATISTICS TIME ON;  
SET STATISTICS IO ON; → this shows the server parse and compile time
```

where the **parse and compile time** reported by SQL Server is treated as a proxy for plan generation time.

2. Experiment 1: Contradiction and Tautology Policies

2.1 Objective

The goal of this experiment is to evaluate how different enforcement mechanisms behave when policies involve **logical contradiction and composition**, specifically:

- ($p(o)$ and $\sim p(o)$)
- ($p(o)$ and $\sim q(o)$)
- ($\sim q(o)$ and $p(o)$)

These cases are derived from the reference paper - Compliance in Databases: A Study of Structural Policies and Query Optimization and are intended to examine:

- Whether redundant or contradictory predicates are optimized away
- The impact on planning and execution time
- Differences in behavior between UDF and RLS implementations

2.2 Design

- Two base policy functions were defined:
 - ($p(o)$): predicate on order attributes
 - ($q(o)$): another predicate on the same relation
- A negation function was constructed:
 - ($\sim q(o)$) implemented as a wrapper UDF
- A **wrapper UDF** was used to compose policies:

$\text{wrapper}(o) = p(o) \text{ AND NOT } q(o)$

- The composed policy was applied in the query as:

```
SELECT COUNT(*)
FROM dbo.orders
WHERE dbo.fn_policy_wrapper_udf_2(...) = 1
OPTION (USE HINT('DISABLE_TSQL_SCALAR_UDF_INLINING'));
```

- Equivalent policies were implemented using RLS for comparison.
- Policy Coverage Indexes (PCI) were created to support predicate evaluation.

2.3 Results

Case 1: (p(o) and ~p(o))

- **UDF**
 - Parse/compile time: ~1–4 ms
 - Execution time: ~12.8 seconds
- **RLS**
 - Parse/compile time: ~7–8 ms
 - Execution time: ~0.14 seconds

Case 2: (p(o) and ~q(o))

- **UDF**
 - Parse/compile time: ~3–5 ms
 - Execution time: ~51.8 seconds
- **RLS**
 - Parse/compile time: ~13 ms
 - Execution time: ~0.11 seconds

Case 3: (~q(o) and p(o))

- **UDF**
 - Parse/compile time: ~0 ms
 - Execution time: ~76.9 seconds
- **RLS**
 - Parse/compile time: ~31 ms
 - Execution time: ~0.14 seconds

2.4 Observations

1. Planning Time

- UDF consistently shows **lower parse and compile time**.
 - This is because the UDF is treated as a **black box**, and its internal logic is not expanded into the execution plan.
 - 2. **Execution Time**
 - UDF execution time is **significantly higher** (orders of magnitude larger).
 - This is due to **row-by-row evaluation**, where the function is invoked for each tuple.
 - 3. **Effect of Logical Composition**
 - Even for contradiction ((p and $\sim$ p)), the UDF approach does not eliminate redundant computation.
 - The function is still evaluated per row, leading to unnecessary work.
 - 4. **Order Sensitivity**
 - The cases (p and $\sim$ q) and ($\sim$ q and p) show different execution times in UDF.
 - This indicates that **evaluation order affects cost** in UDF-based execution.
 - 5. **RLS Behavior**
 - RLS achieves significantly lower execution time.
 - This suggests that predicates are evaluated in a **set-oriented manner**, allowing efficient pruning.
-

3. Experiment 2: Cyclic Policy Queries

3.1 Objective

This experiment evaluates the behavior of policy enforcement mechanisms on **cyclic query-policy mappings**, as defined in the referenced repository https://github.com/imtiyazuddin/secure26db/blob/main2/src/postgreSQL/cyclic_policy_query_mapping.json.

The goal is to study:

- Performance under cyclic dependencies
- Differences across UDF, RLS, and view-based implementations
- Impact on planning and execution time

3.2 Design

- Queries: 3, 12, 17, and 18
- Policies were identified from the JSON specification in the referenced repository
- The corresponding PostgreSQL policies were **manually translated into T-SQL**
- For each query:
 - Policies were implemented in three forms:
 - RLS (security policies)
 - View-based filtering
 - UDF-based filtering
- The policies were then **manually applied** to the relevant tables for each query configuration
- Each query was executed under different policy implementations, and planning and execution times were recorded using:

SET STATISTICS TIME ON;
SET STATISTICS IO ON;

3.3 Results

Query 3

- **View**
 - Compile: ~144 ms
 - Execution: ~0.96 s
- **UDF**
 - Compile: ~38 ms
 - Execution: ~77.4 s
- **RLS**
 - Compile: ~197-199 ms
 - Execution: ~0.6 s

Query 12

- **View**
 - Compile: ~26 ms
 - Execution: ~0.45 s
- **UDF**
 - Compile: ~4–5 ms
 - Execution: ~27.1 s
- **RLS**
 - Compile: ~26-26 ms
 - Execution: ~0.13 s

Query 17

- **View**
 - Compile: ~108 ms
 - Execution: ~0.26 s
- **UDF**
 - Compile: ~19–21 ms
 - Execution: ~0.43 s
- **RLS**
 - Compile: ~79 ms
 - Execution: ~0.19 s

Query 18

- **View**
 - Compile: ~169 ms
 - Execution: ~1.37 s
- **UDF**
 - Compile: ~35–37 ms
 - Execution: ~1.30 s
- **RLS**

- Compile: ~188-191 ms
- Execution: ~1.35 s

3.4 Observations

1. Planning Time Trends

- UDF consistently shows the **lowest compile time** across all queries.
- This is due to its **black-box nature**, where internal logic is not expanded into the execution plan.
- View-based approaches show **moderate compile time**, as their logic is visible but structured.
- RLS shows the **highest planning time**, as predicate functions are integrated into the query plan and require more optimization effort.

2. Execution Time Trade-off

- Lower compile time (UDF) does not imply better performance.
- UDF shows **significantly higher execution time** in Queries 3 and 12, indicating inefficiency due to repeated evaluation.
- RLS generally provides **stable and efficient execution**, especially in Queries 12 and 17.

3. Query Sensitivity

- Performance varies significantly depending on query structure:
 - i. Query 17 shows comparable performance across all approaches.
 - ii. Query 18 shows similar execution times for View and UDF, while RLS incurs higher cost.
- This indicates that **query shape and join complexity** strongly influence performance

4. Black-Box Effect (UDF)

- UDF consistently exhibits **low compile time** due to lack of optimizer visibility.
 - However, this leads to:
 - i. No predicate pushdown
 - ii. No reuse of intermediate results
 - iii. Repeated evaluation per row
 - As a result, execution time increases significantly for complex queries.
-

4. Overall Observations

- **UDF-based enforcement**
 - Lowest planning time
 - Highest execution time in complex queries
 - Strongly affected by predicate composition and ordering
 - Inefficient due to row-by-row evaluation
- **RLS-based enforcement**
 - Highest planning time
 - Generally better execution performance
 - More stable across different query types
 - Benefits from integrated, set-based evaluation
- **View-based enforcement**
 - Moderate planning time

- Performs better than UDF in most cases
-

5. Key Insight

Treating policy logic as a black box (UDF) reduces planning overhead but leads to significant execution cost due to lack of optimization and repeated evaluation.

PLOTS TO BE ADDED FOR THE RESULTS (FOR EASY TIME COMPARISONS)

References

Killing long running queries

https://www.w3tutorials.net/blog/how-to-find-current-long-running-queries-in-sql-server-and-how-to-kill-them-instantly/#google_vignette

Adaptive join - <https://sqlserverfast.com/epr/adaptive-join/>

Hash join and nested loop join -

<https://www.bing.com/videos/riverview/relatedvideo?q=Join+algorithms+andy+pavlo&&mid=3E6A5E6654EF3D4A98A73E6A5E6654EF3D4A98A7&FORM=VAMGZC>